

DEPARTAMENTO
DE COMPUTACION

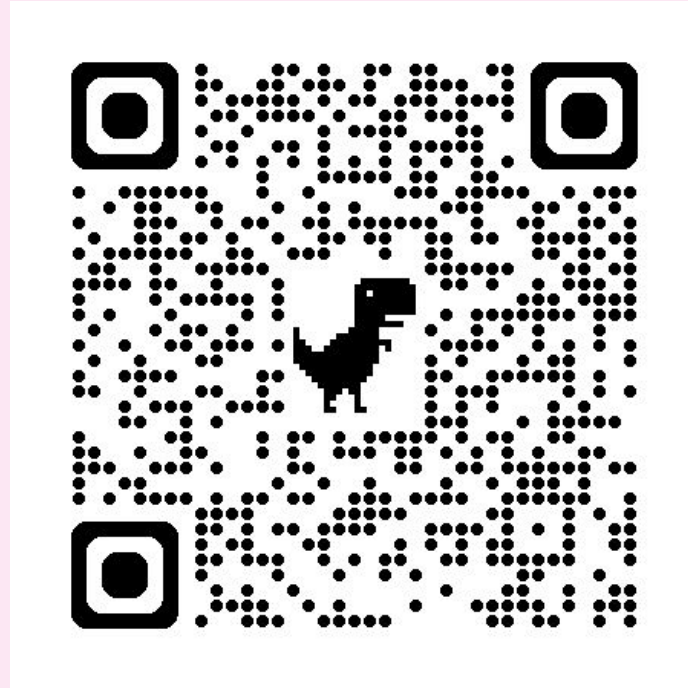
Facultad de Ciencias Exactas y Naturales - UBA

Construyendo Agentes de Inteligencia Artificial

Día 3: Agentes - Frameworks, ADK

Miércoles 29 de Julio del 2026

Repaso Clase 2



**Agentes que me permiten
escribir código**

**y hacer subagentes que me
ayuden a escribir código**

**(de cualquier tipo: java, c++,
go, html, python, incluso
Agentes en LangChain o ADK)**



Ejemplos de Agentes

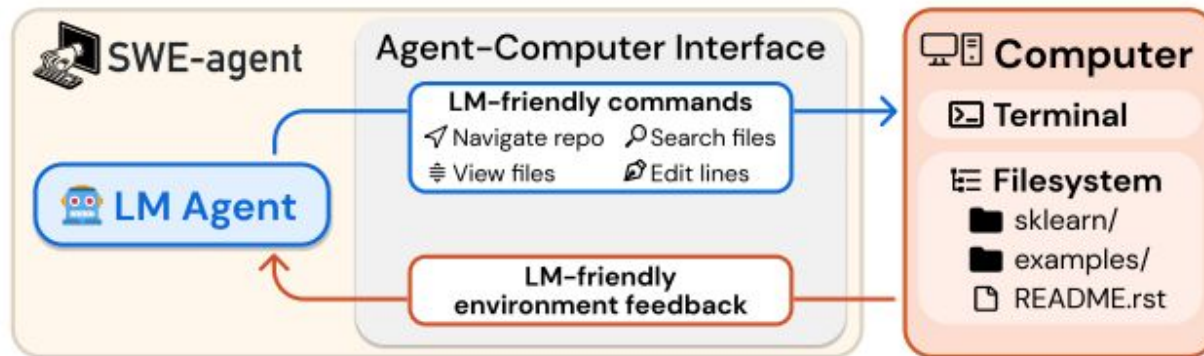
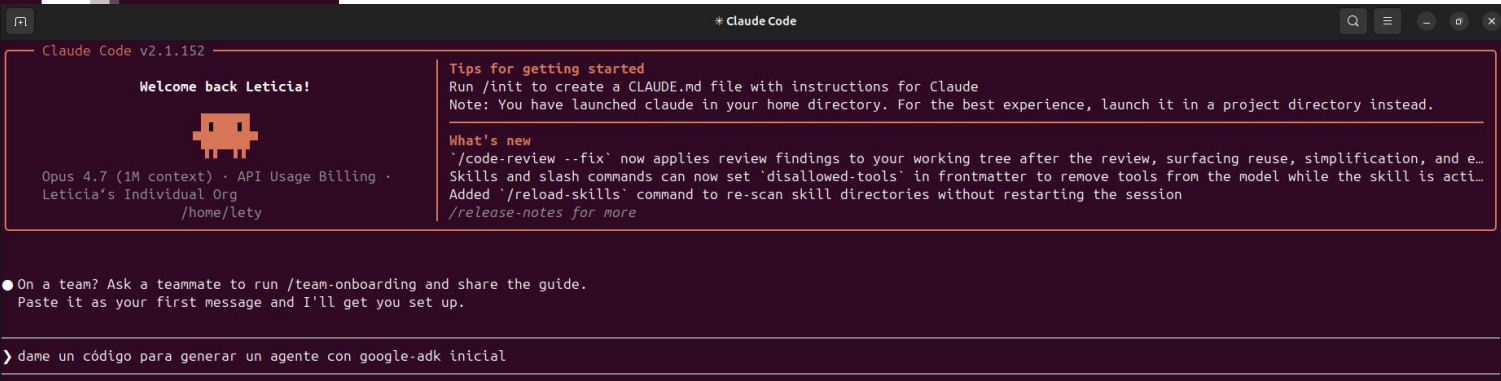
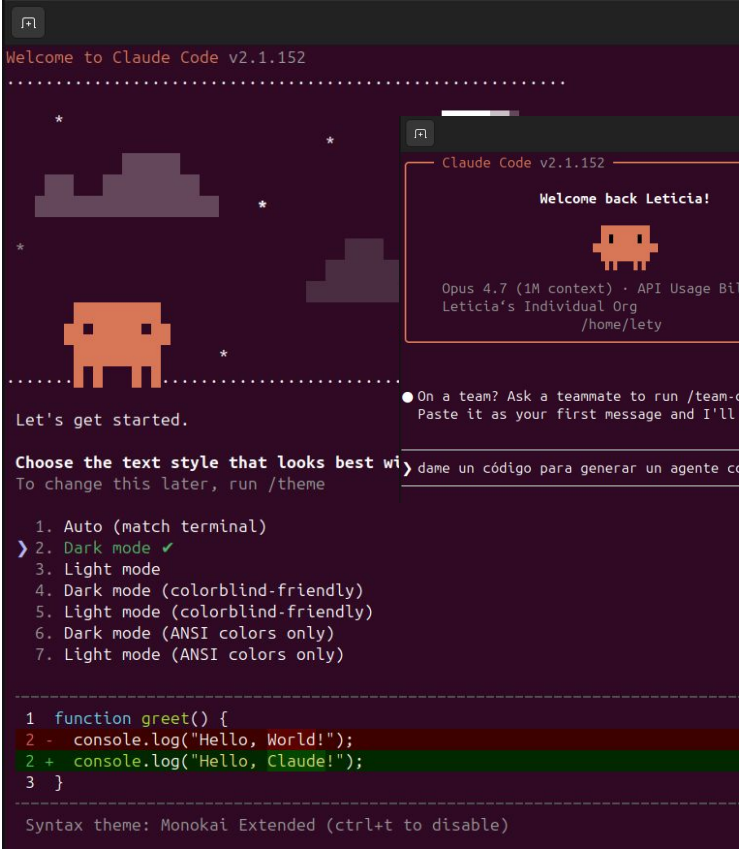


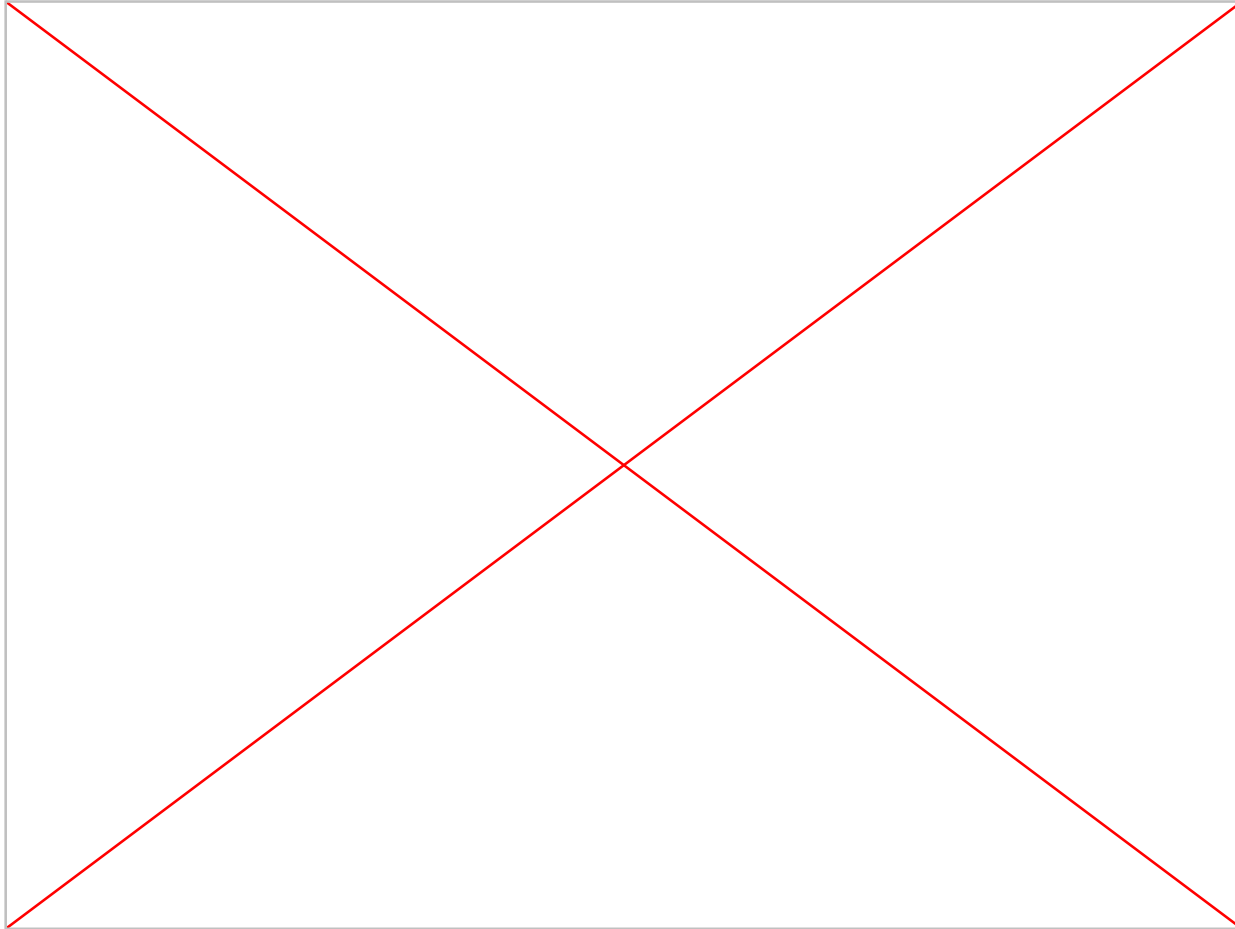
Figure 1: SWE-agent is an LM interacting with a computer through an agent-computer interface (ACI), which includes the commands the agent uses and the format of the feedback from the computer.

SWE-agent <https://arxiv.org/pdf/2405.15793>

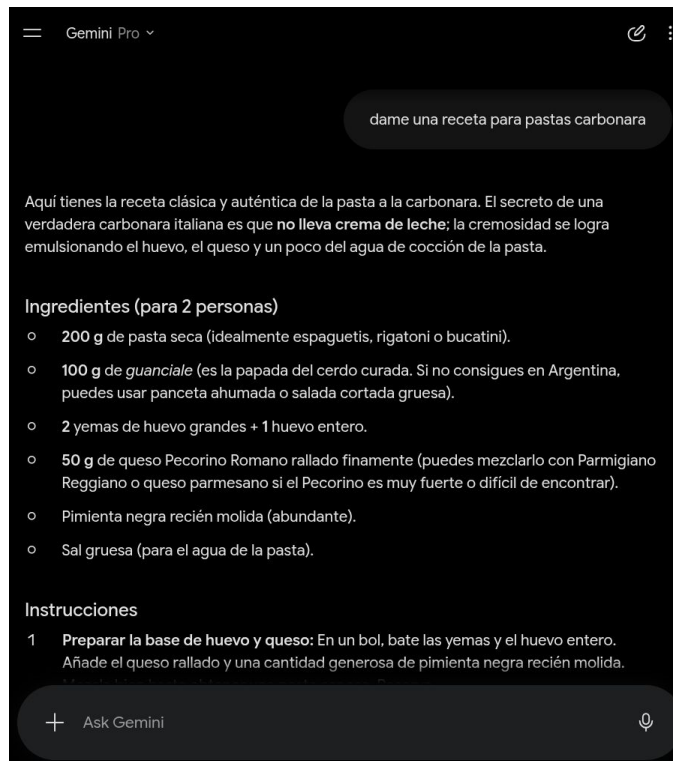
Ejemplos de Agentes: Claude Code



Docs: <https://code.claude.com/docs/en/overview>

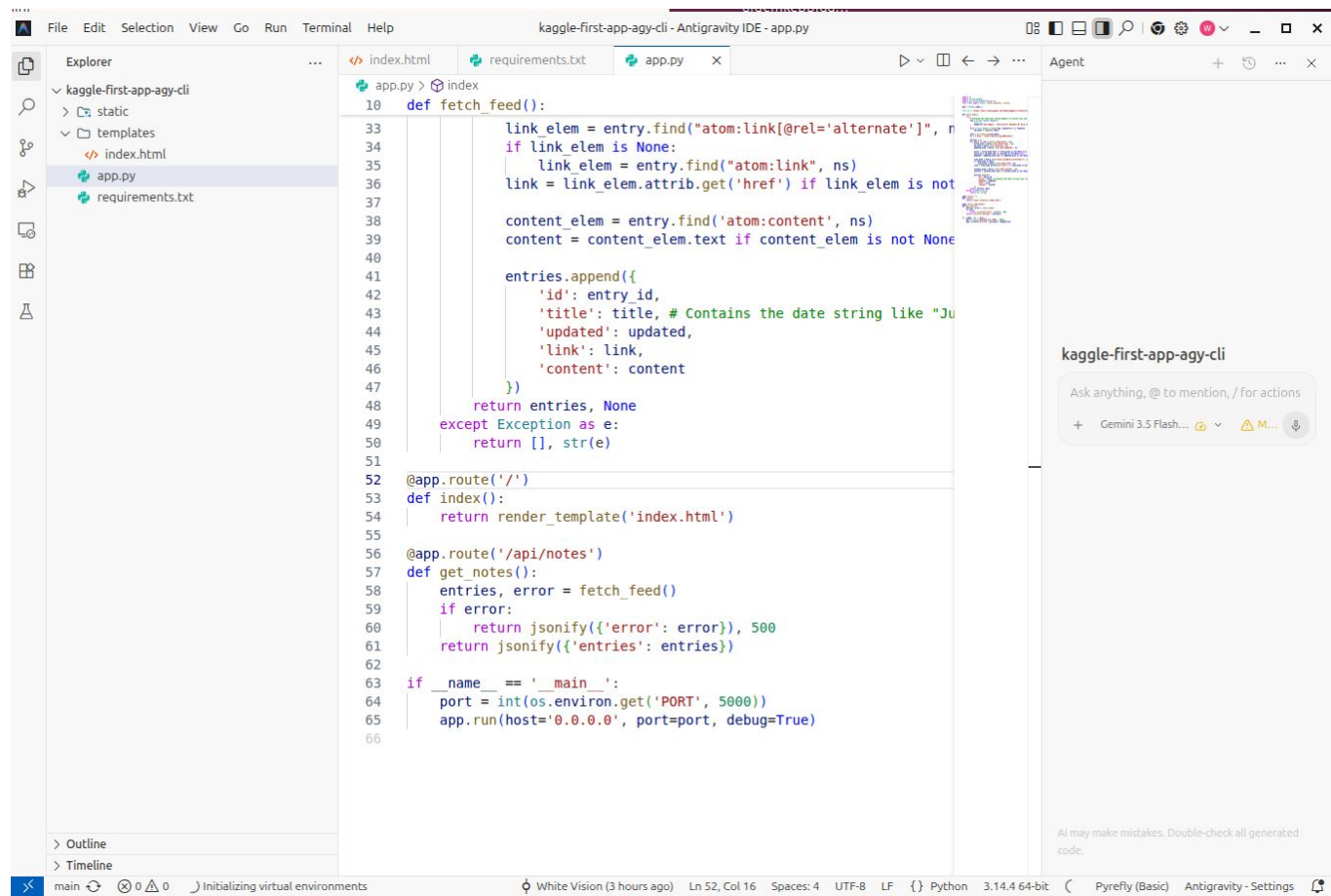


Ejemplos de Agentes: Gemini Gemini chat

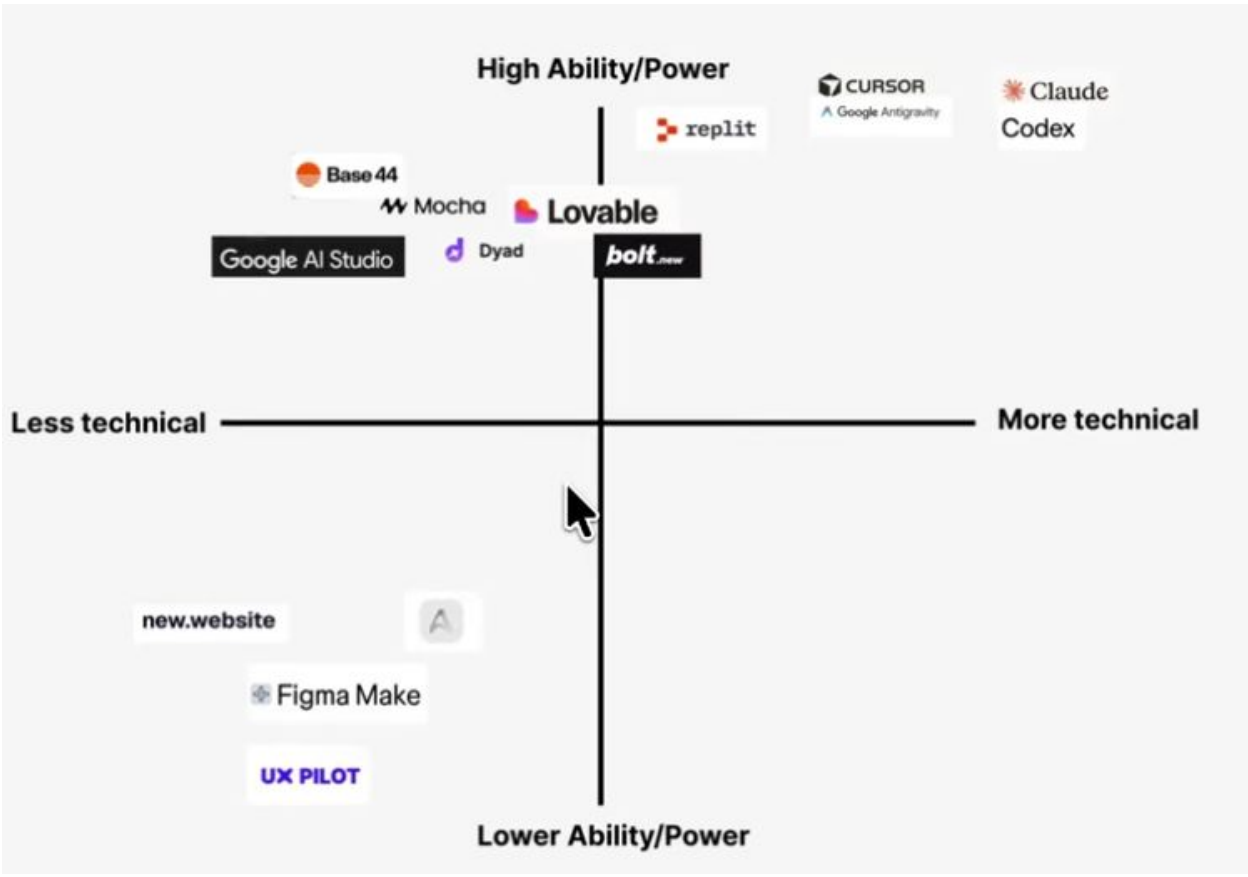


Gemini app: <https://gemini.google.com/app>

Google Antigravity IDE



Agentes para Programar (Codear)



Librerías para codear Agentes

<https://adk.dev/get-started/>

Agentes RAG: Framework: LlamaIndex

LlamaIndex proporciona las herramientas para construir casos de uso de **aumento de contexto**, desde el prototipo hasta la producción.

Cuenta con herramientas que permiten:

- ingerir,
- parsear,
- indexar,
- procesar datos,
- implementar rápidamente flujos de trabajo de consulta complejos combinando el acceso a los datos con el *prompting* de LLMs.
- armar agentes

El ejemplo más popular de aumento de contexto es la Generación Aumentada por Recuperación o RAG (por sus siglas en inglés), que combina el contexto con los LLMs en tiempo de inferencia.

<https://www.llamaindex.ai/>

<https://developers.llamaindex.ai/llamaparse/>

Agentes: Framework: LangChain

- Open-source framework para crear agentes
- Simplifica el desarrollo, despliegue y puesta en producción de aplicaciones que usen Modelos de Lenguaje (LLMs)
- Funcionalidades:
 - LangChain Expressions Language (LCEL) para componer y manejar cadenas (chains)
 - LangGraph para crear aplicaciones multiagentes
 - LangServe para crear APIs REST
 - LangSmith una plataforma de desarrollo para debuggear, testear y monitorear aplicaciones de LLMs
 - Integración con fuentes de datos para RAG

Agentes: Framework: Google ADK

- Open-source framework para crear agentes
- Creado por Google
- Ayuda en la creación, gestión, evaluación y despliegue de agentes impulsados por Inteligencia Artificial

Discover, build and deploy agents

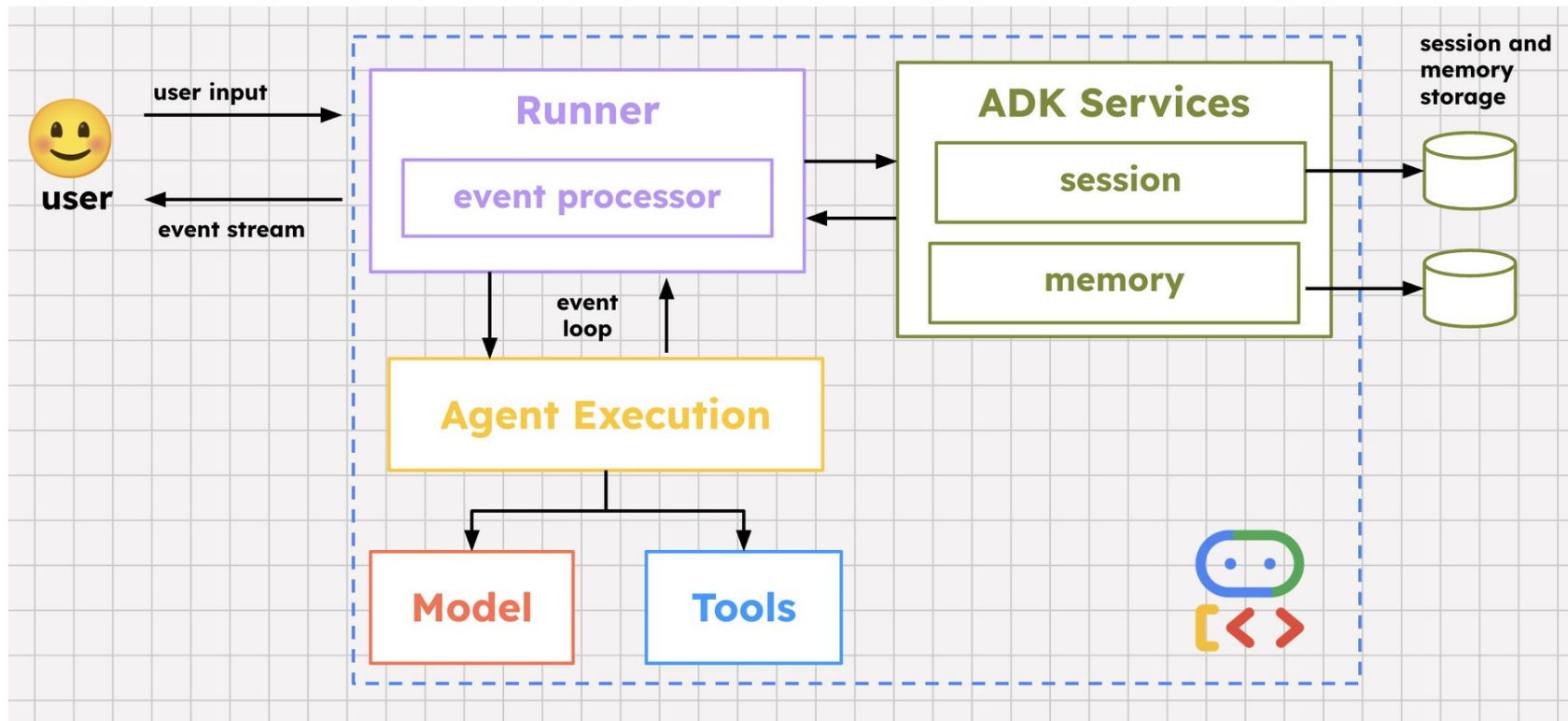


<https://adk.dev/get-started/about/>

Agentes: Frameworks - Comparativo

Herramienta / Framework	Enfoque Principal	Fortaleza Clave	Manejo de Datos	Flujo de Trabajo	Complejidad
ADK (Agentic Dev Kit)	Creación rápida de Agentes.	Abstracción de alto nivel para herramientas.	Utiliza los motores de LlamaIndex.	Basado en Tasks y Tools.	Baja/Media (Diseñado para rapidez).
CrewAI	Orquestación de sistemas multi-agente colaborativos.	Agentes con roles definidos que colaboran entre sí (Role-playing).	Delegado a herramientas (Se integra nativamente con LangChain/LlamaIndex).	Basado en Agents, Tasks y Crews (Procesos secuenciales o jerárquicos).	Media (Simplifica la lógica compleja de múltiples agentes interactuando).
LlamaIndex	Datos y Recuperación (RAG).	Indexación y búsqueda avanzada.	Excelente (Estructura datos complejos).	Basado en Query Engines.	Media (Fácil de empezar para RAG).
LangChain	Orquestación y Cadenas.	Ecosistema masivo e integraciones.	Genérico (Carga básica de documentos).	Basado en Chains y DAGs.	Alta (Mucha flexibilidad, curva lenta).
LangGraph	Flujos de trabajo cíclicos y agentes con estado (Stateful).	Ciclos (loops), memoria persistente y control detallado del flujo paso a paso.	Agnóstico (Basado en la actualización y paso de un "Estado" entre nodos).	Basado en Grafos (Nodos, Aristas y Estados condicionales).	Alta (Curva de aprendizaje empinada, ideal para control total sobre agentes).

Agentes: Frameworks - Google ADK (Agent Development Kit)



Google ADK <https://adk.dev/get-started/python/>
<https://cloud.google.com/blog/topics/developers-practitioners/remember-this-agent-state-and-memory-with-adk>

Componentes de Google ADK



Agente (Agent)

La unidad fundamental de trabajo diseñada para tareas específicas. Los agentes pueden usar modelos de lenguaje (LlmAgent) para el razonamiento complejo, o actuar como controladores deterministas de la ejecución, a los cuales se les llama "agentes de flujo de trabajo".



Herramienta (Tool)

Otorga a los agentes habilidades más allá de la conversación, permitiéndoles interactuar con APIs externas, buscar información, ejecutar código o invocar otros servicios externos al modelo base.



Memoria (Memory)

Permite a los agentes recordar información sobre un usuario a través de múltiples sesiones, proporcionando un contexto a largo plazo persistente (distinto del Estado de sesión a corto plazo).



Modelos

El LLM base que impulsa a los LlmAgents, procesando los tokens y habilitando las capacidades abstractas de comprensión, generación y razonamiento lógico.



Callbacks

Fragmentos de código personalizado que proporcionas para que se ejecuten en puntos específicos del proceso del agente, lo que permite realizar verificaciones, registros (logging) o modificaciones dinámicas de comportamiento.



Planificación (Planning)

Una capacidad cognitiva avanzada donde los agentes pueden desglosar objetivos de alto nivel en pasos lógicos más pequeños, evaluar rutas alternativas y organizar un cronograma de ejecución (ej. arquitecturas ReAct).

Componentes de Google ADK



Gestión de Sesiones

Maneja el contexto de una conversación individual (**Session**), incluyendo su historial (**Eventos**) y la memoria de trabajo temporal del agente para esa conversación (**Estado**).



Artefactos (Artifact)

Permite a los agentes guardar, cargar y gestionar archivos o datos binarios complejos (como imágenes, documentos PDFs, hojas de cálculo) asociados con una sesión o usuario particular.



Ejecución de Código

La capacidad de los agentes (generalmente a través de Herramientas especializadas) para generar, probar y ejecutar código en entornos seguros (sandbox) con el fin de realizar cálculos, análisis de datos o acciones complejas.



Evento (Event)

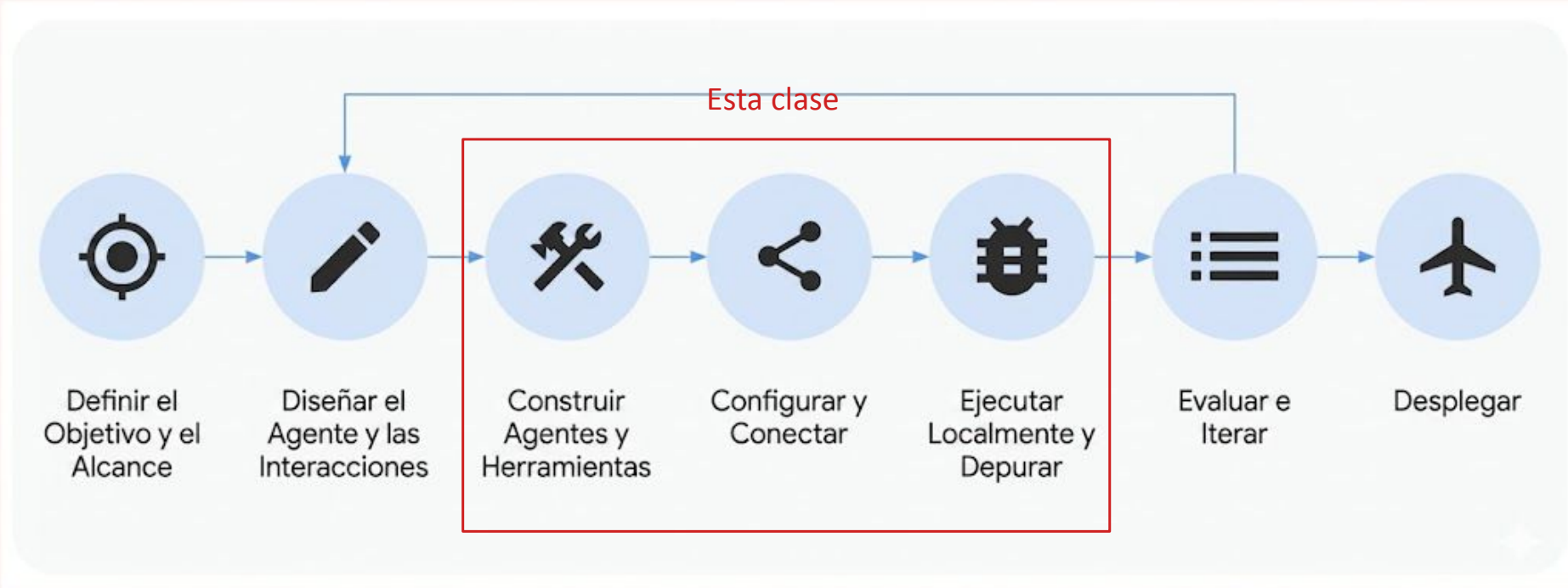
La unidad atómica de comunicación. Representa sucesos asíncronos en una sesión (inputs del usuario, outputs del agente, fallos de red), construyendo el grafo de la conversación e instruyendo la dirección del sistema.



Ejecutor (Runner)

El motor principal (Loop de control) que gestiona el flujo de ejecución, enruta los Eventos, orquesta la interacción entre múltiples agentes y se comunica con la infraestructura de backend de manera continua.

Proceso de Creación de Agentes

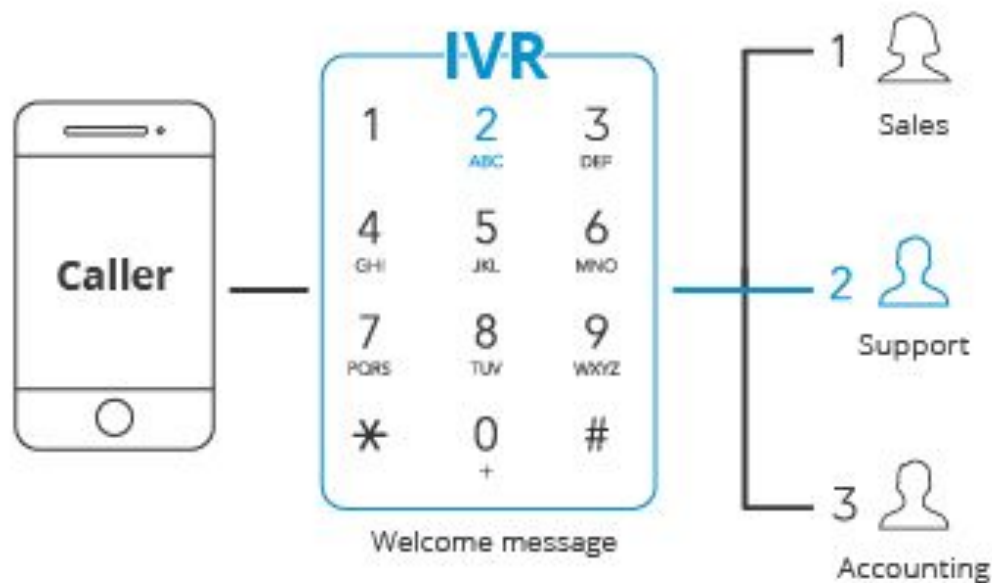


Ejemplos de Agentes: Chatbot Whatsapp



Fuente: <https://verge-ai.com/whatsapp-chatbot/>

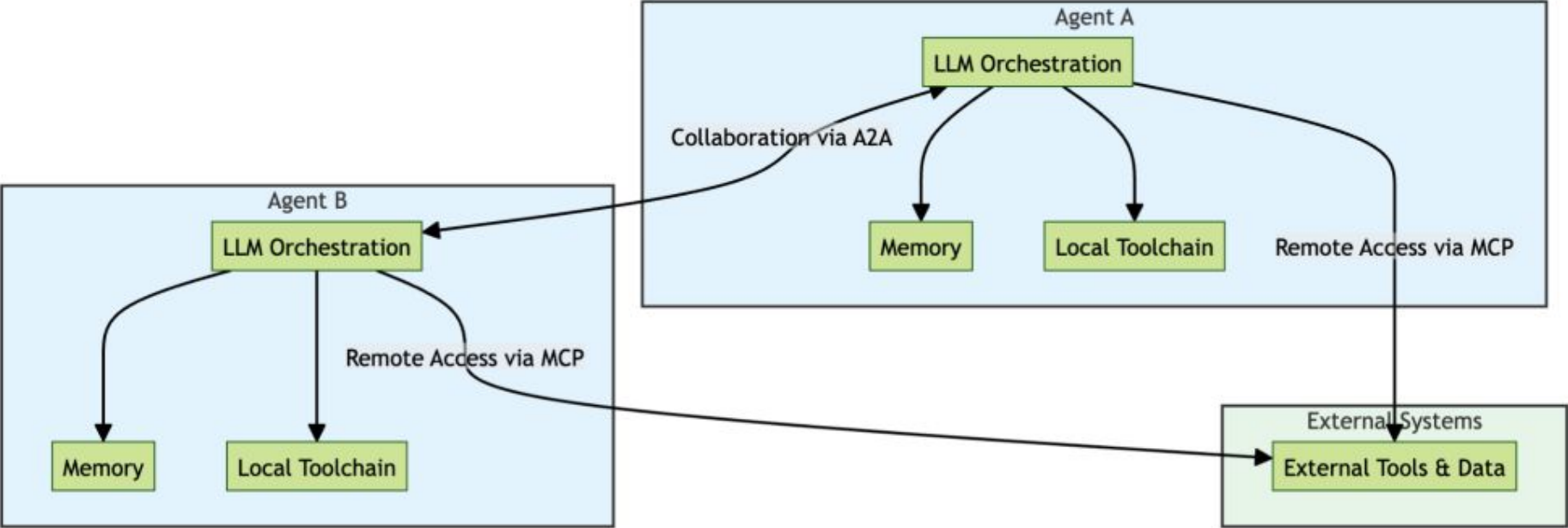
Ejemplos de Agentes: Evolución IVR (Interactive Voice Response)



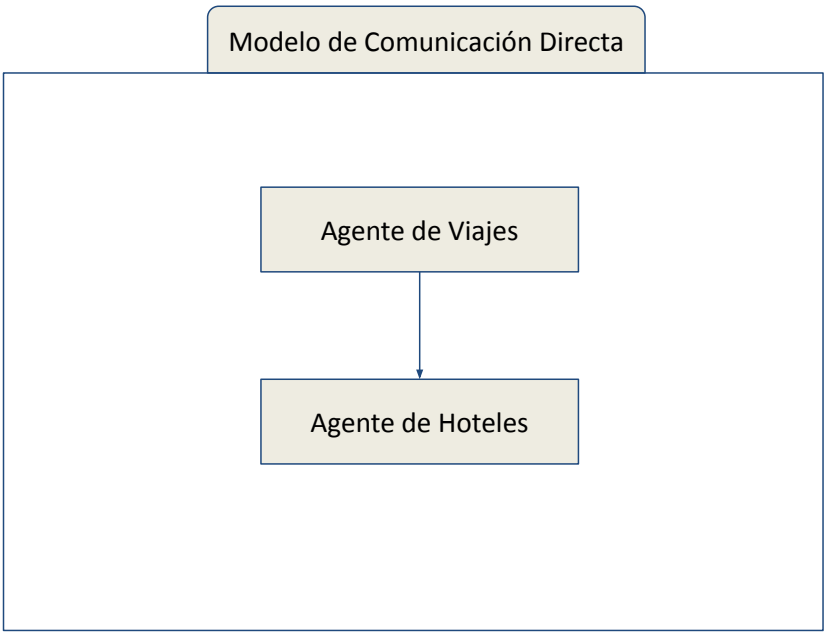
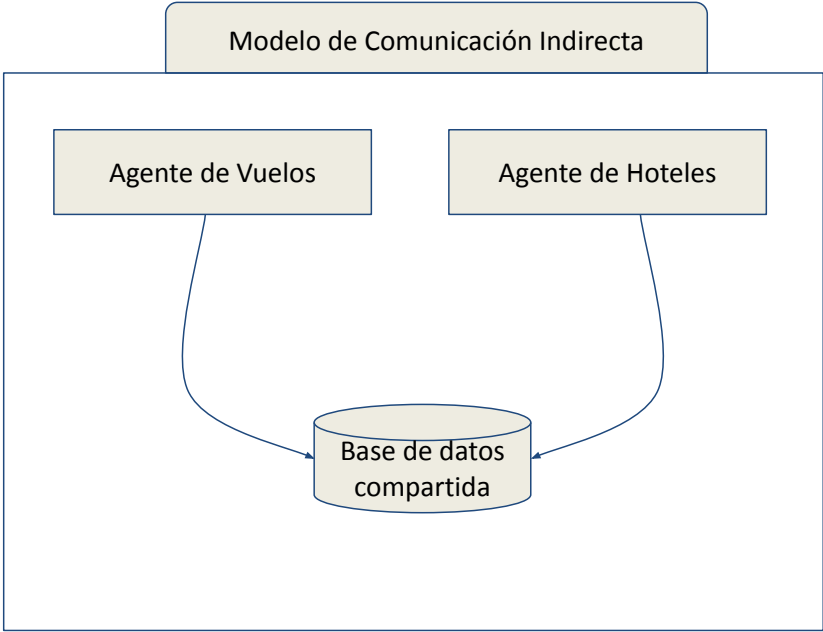
Los IVR utilizan NLP (Natural Language Processing) para determinar la intención del usuario. El surgimiento de los Agentes híbridos permite la creación de aplicaciones híbridas para la atención al cliente o atender llamadas. Surgen así nuevas plataformas de Agentes Conversacionales por voz.

Arquitecturas Multiagentes

Arquitectura Multi-Agente

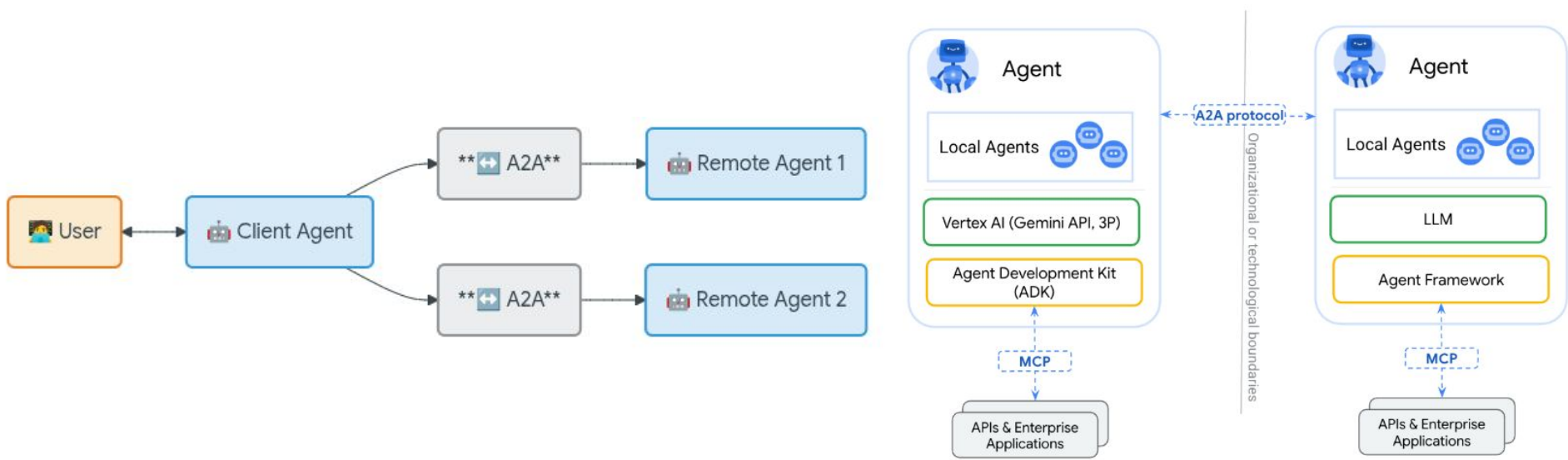


Interacción entre Agentes



A2A

A2A Agent to Agent - Se trata de un protocolo que permite la comunicación entre Agentes. Esto es particularmente útil si tenemos Agentes desarrollados con distinta tecnología y precisan comunicarse entre sí.

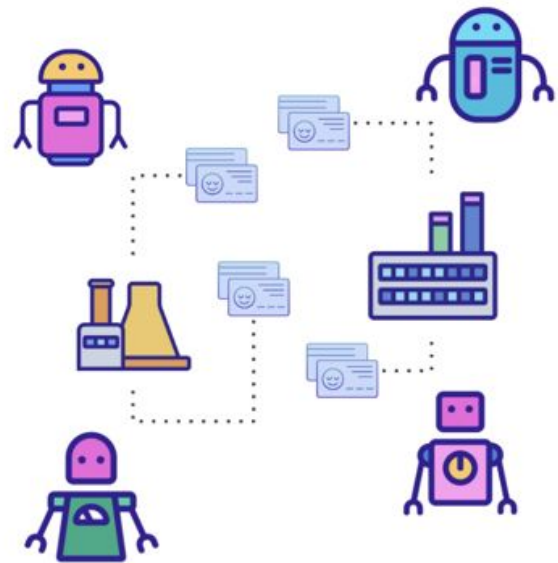


Agent-to-Agent <https://a2a-protocol.org/latest/>

A2A - Agent Card



A2A - Agent Card



Name: Agent ABC
Provider: Google (URL)
Preferred input/output: text

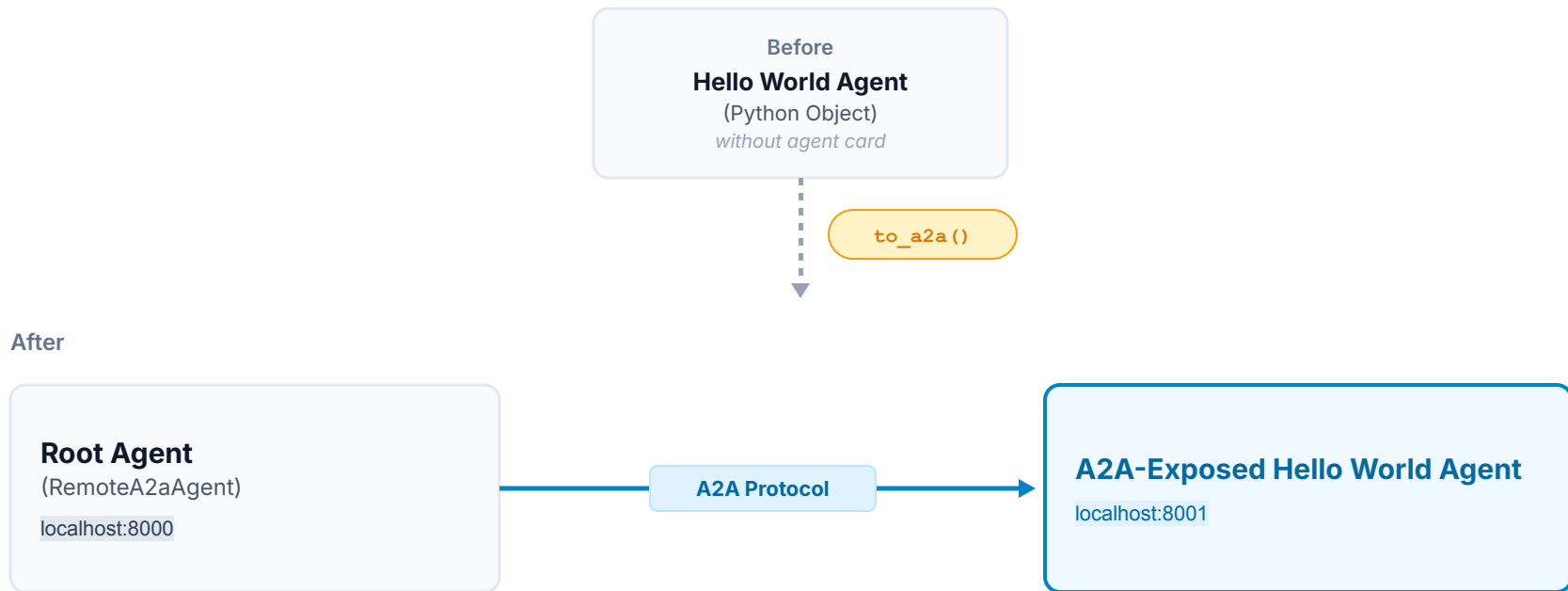
Capabilities:
I can do streaming
I can do Push Notifications

- Skills:**
- Skill A
 - Description
 - Input
 - Output
 - Skill B
 - Description
 - Input
 - Output

Authentication:
Scheme: Bearer
Key: xxx

<https://codelabs.developers.google.com/codelabs/currency-agent#0>

A2A - Google ADK



Documentation: <https://adk.dev/a2a/quickstart-exposing/>

Demo

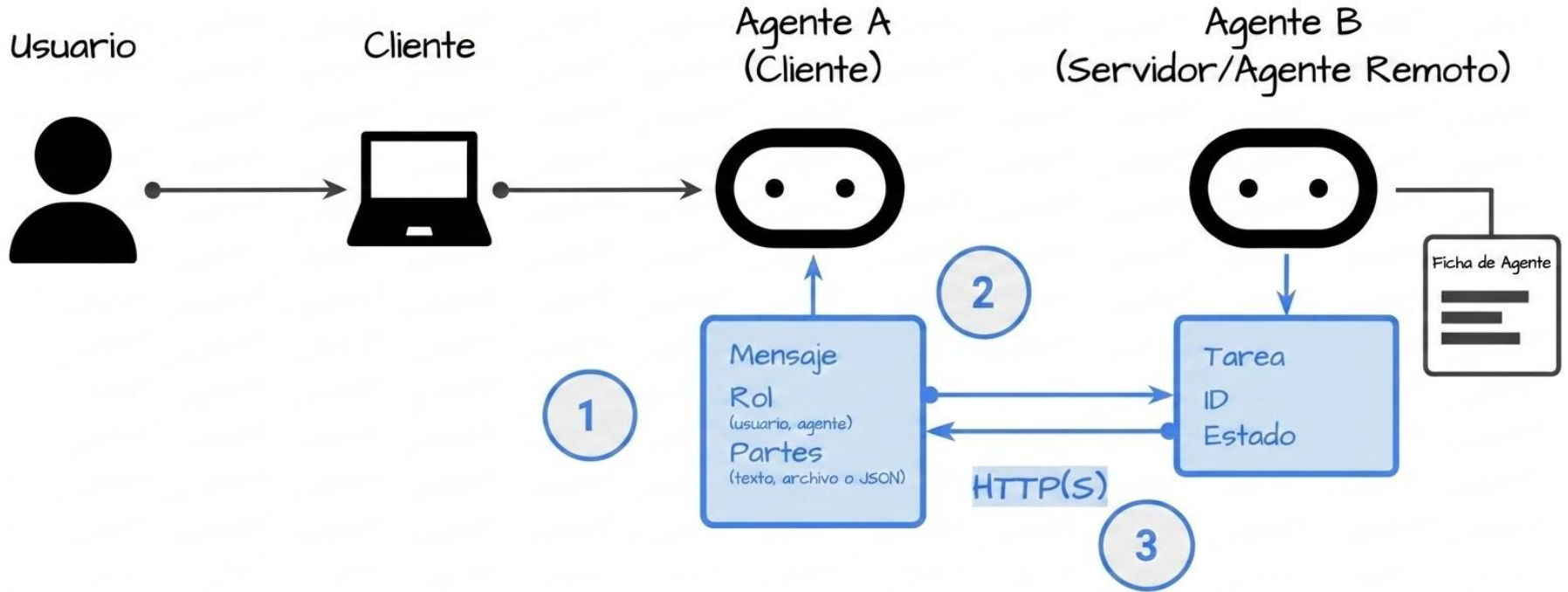
A2A - Agent-to-Agent

A2A - Librería: Conceptos Clave

Elemento	Descripción	Propósito Clave
Ficha de Agente Agent Card	Un documento de metadatos JSON que describe la identidad, capacidades, endpoint, habilidades y requisitos de autenticación de un agente.	Permite a los clientes descubrir agentes y comprender cómo interactuar con ellos de manera segura y efectiva.
Tarea Task	Una unidad de trabajo con estado iniciada por un agente, con un ID único y un ciclo de vida definido.	Facilita el seguimiento de operaciones de larga duración y permite colaboraciones e interacciones de múltiples turnos.
Mensaje Message	Un único turno de comunicación entre un cliente y un agente, que contiene contenido y un rol ("usuario" o "agente").	Transmite instrucciones, contexto, preguntas, respuestas o actualizaciones de estado que no son necesariamente formales.
Parte Part	El contenedor de contenido fundamental en Mensajes y Artefactos. Contiene texto, referencia de archivo (URL o bytes) o datos estructurados.	Proporciona flexibilidad a los agentes para intercambiar diversos tipos de contenido dentro de mensajes y artefactos.
Artefacto Artifact	Un resultado tangible generado por un agente durante una tarea (por ejemplo, un documento, imagen o datos estructurados).	Entrega los resultados concretos del trabajo de un agente, garantizando salidas estructuradas y recuperables.

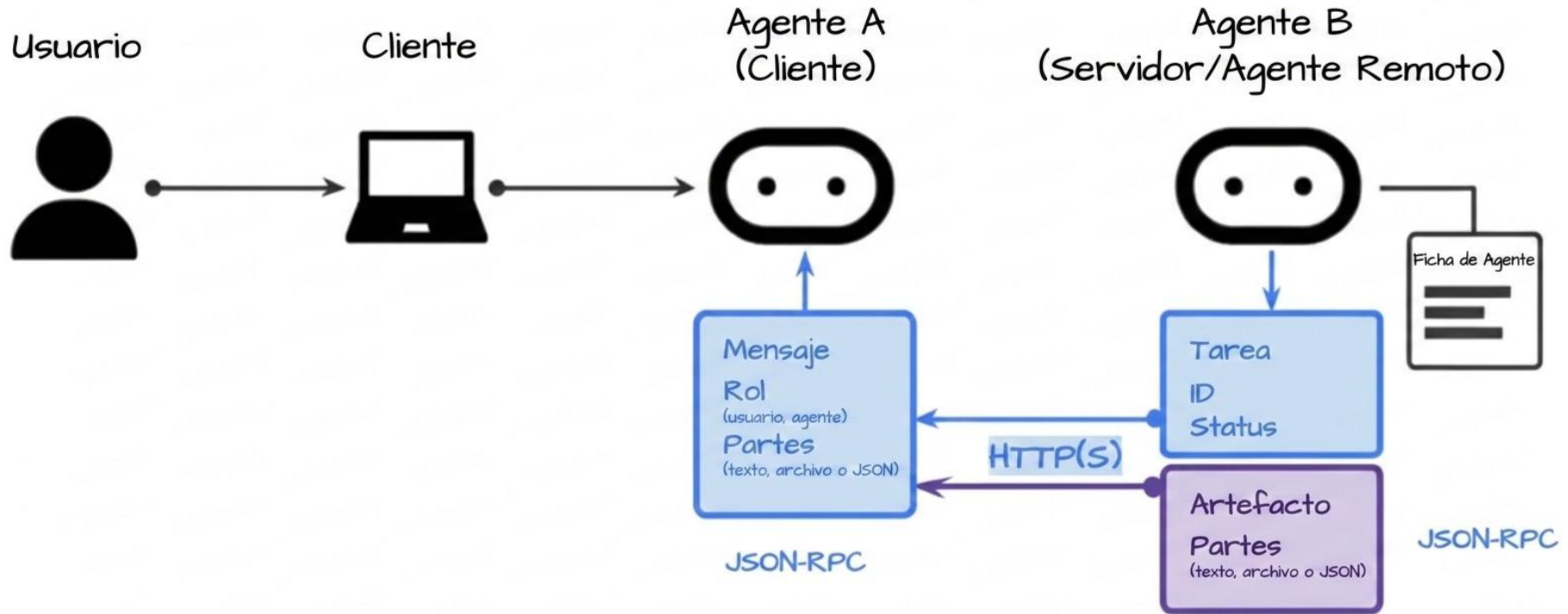
Documentación: <https://a2a-protocol.org/latest/topics/key-concepts/>

A2A - Comunicación - Polling

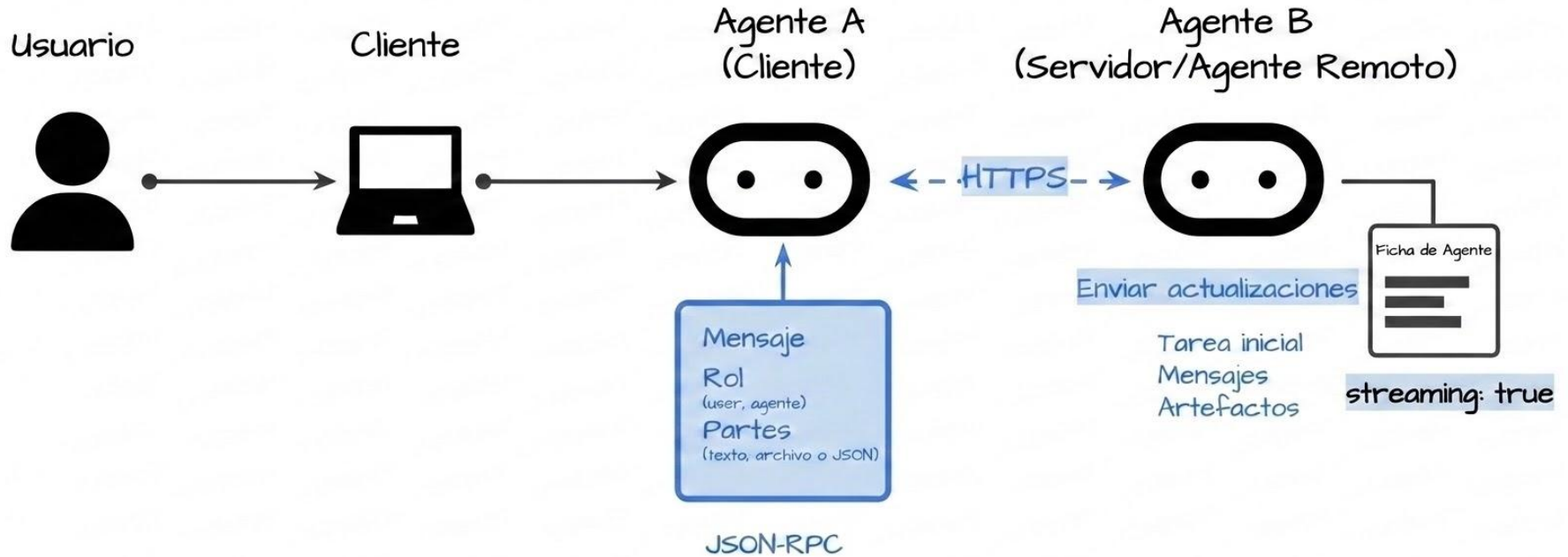


Como funciona A2A Video: https://www.youtube.com/watch?v=Fbr_Solax1w

A2A - Comunicación - Resultado



A2A - Comunicación - Streaming



<https://a2a-protocol.org/latest/tutorials/python/1-introduction/>
<https://docs.langchain.com/langsmith/server-a2a>

Demo

A2A - Agent-to-Agent

2

Niveles de Maduración

Podemos clasificar los Agentes desde distintas perspectivas, sin embargo, en este apartado vamos a clasificarlos según su complejidad.

A mayor autonomía y complejidad del sistema agéntico, mayores son los riesgos y controles que se precisan. También, va a estar relacionado con los casos de uso que busquemos resolver, un agente simple que se dedica a diagnosticar personas sin duda requerirá muchísimo control.

Definimos 6 niveles que iremos viendo a lo largo del curso:

1. Sistema básico agéntico (single agent)
2. Sistema dinámico de workflows de agentes (dynamic workflow agents)
3. Sistema agéntico con patrones introspectivos como React y Reflexion (react and reflexion agents)
4. Sistemas multi-agénticos (multi-agent)
5. Sistemas multiagénticos avanzados con meta-agentes (Advanced multi-agent)
6. Sistemas agénticos con mecanismos de feedback y auto-aprendizaje (Self-correcting agents)

Otras formas de clasificar patrones:

<https://docs.cloud.google.com/architecture/choose-design-pattern-agentic-ai-system>

Nivel 4: Sistemas multi-agénticos (multi-agent)

multi-agent

semi-autónomo

Múltiples agentes especializados colaborando para llevar a cabo tareas complejas. Cada agente se focaliza en diferentes tareas no superpuestas. Su organización puede permitir ejecutar tareas en paralelo.

Escalabilidad: Ideal para entornos altamente escalables. Sus tareas se pueden distribuir en múltiples agentes, permitiendo el procesamiento paralelo de flujos complejos.

Complicaciones: Más complejo de manejar. Sistemas de monitoreo son requeridos para asegurarse de que todo el sistema semiautónomo de agentes colabore en una forma que se adhiera a las regulaciones

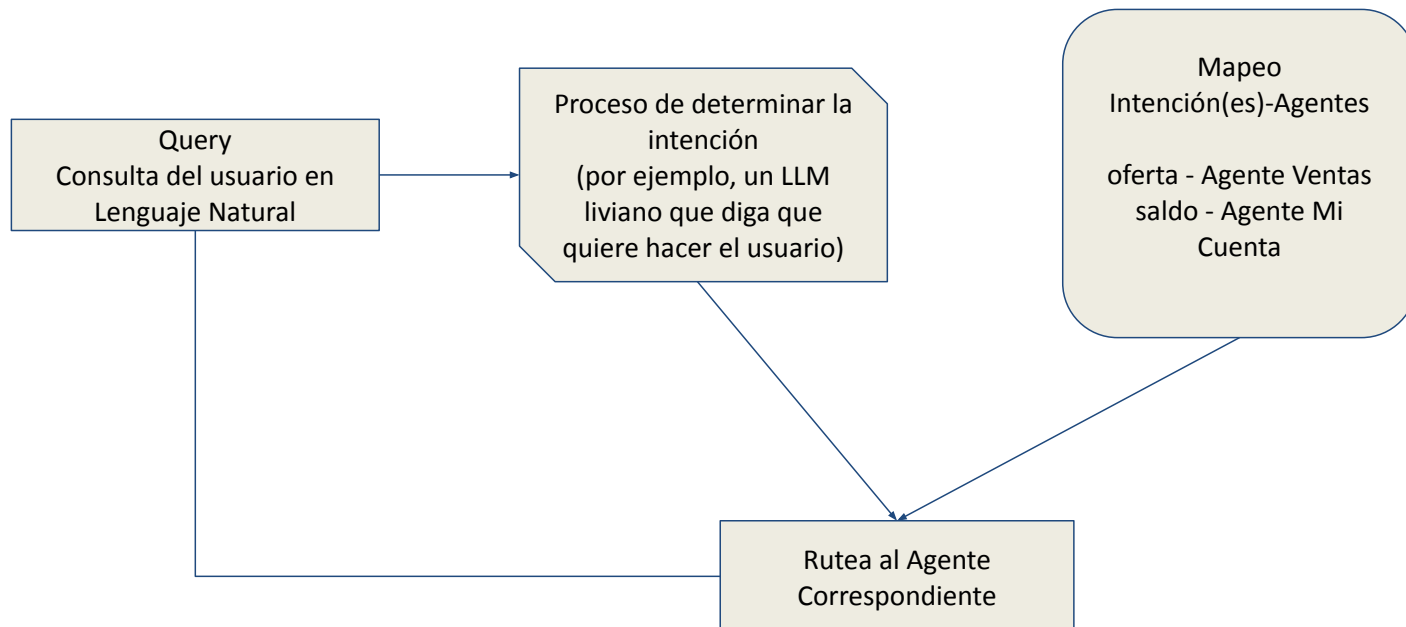
Implementación (Patrones):

- [Arquitectura Supervisor](#)
- [Planificación Multi-Agente \(multi-agent planning\)](#)
- [Shared Epistemic Memory](#)
- [Event Driven Reactivity](#)
- [Tool/Agent Registry](#)

Agent Router

Agent Router pattern
(ruteo basado de la intención)

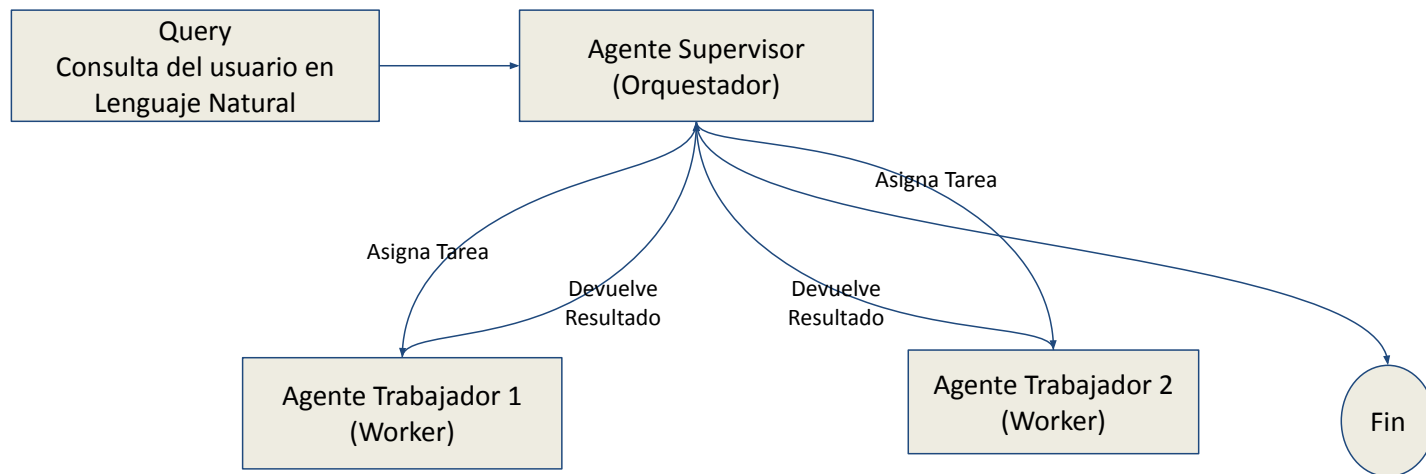
El agente detecta la intención (lo que quiere hacer el usuario) y redirecciona al agente que va a ejecutar lo necesario.



Supervisor Architecture

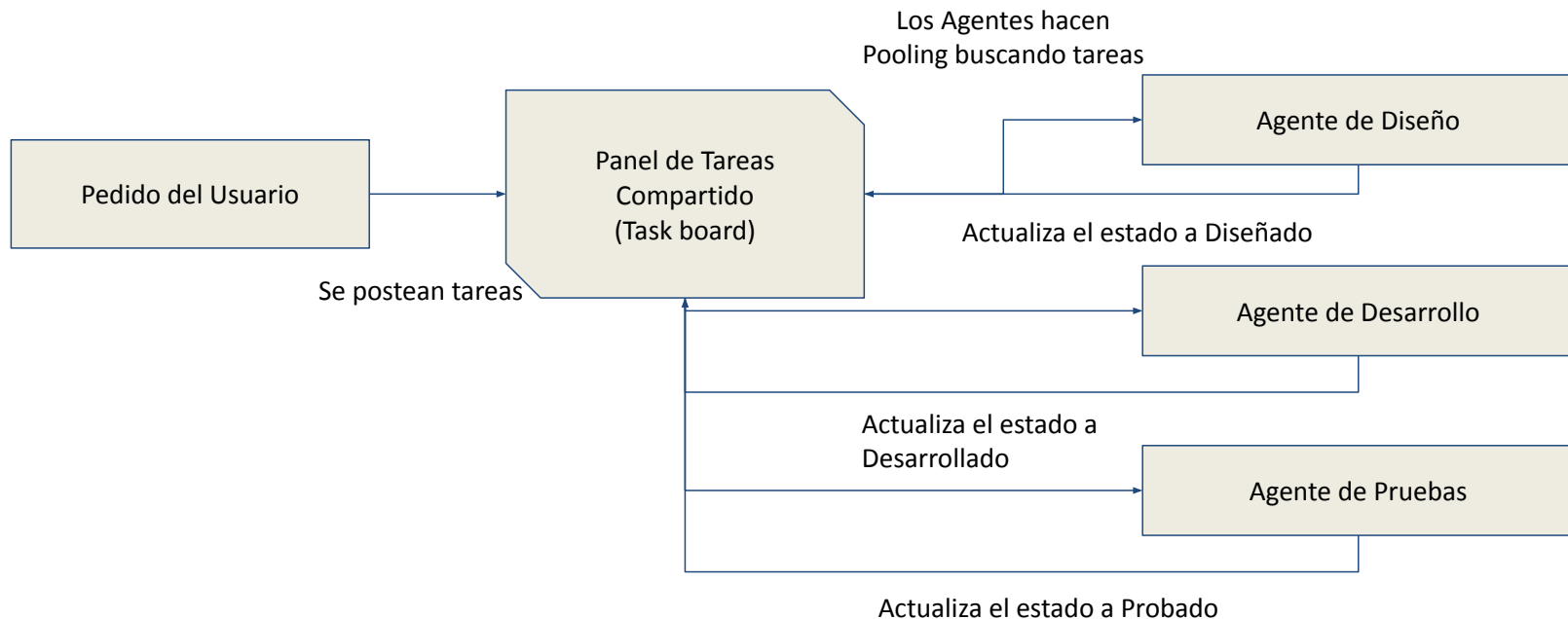
Un agente central que delega en el resto de los agentes.

Contexto: Una tarea compleja requiere la ejecución de muchos pasos secuenciales y el sistema necesita garantizar que el flujo se ejecute correctamente.



Swarm Architecture

Se basa en comunicación compartida o un panel de tareas. Una tarea se poste, y cualquier agente la toma y trabaja en ella. Una vez que el agente termina, actualizar el estado de la tarea, disponibilizándose al próximo agente en el workflow.



Nivel 5: Sistemas multiagénticos avanzados con meta-agentes (Advanced multi-agent)

multi-agent

meta-agents

semi-autónomo

Un “meta-agente” es introducido para supervisar y coordinar a los otros agentes. Esto permite que haya una reasignación dinámica de tareas y ajuste del plan en tiempo real.

Escalabilidad: adaptabilidad mejorada. El sistema puede escalar efectivamente incluso en entornos cambiantes debido a la optimización en la distribución de las tareas por el meta-agente

Complicaciones: Meta-agentes actúan como supervisores, ayudando a mantener la adherencia a la política ajustando los flujos de trabajo (workflows) y robusteciendo las tareas necesarias.

Implementación (Patrones):

- Meta-agents
- Blackboard Topology
- Resource Allocation
- Contract-Net Marketplace
- Supervision Trees

Nivel 6: Sistemas agénticos con mecanismos de feedback y auto-aprendizaje (Self-correcting agents)

multi-agent

feedback loops

self-correction

Avanzados sistemas multiagentes usan feedback loops complejos y de turnos múltiples. Los agentes critican, corrigen y redefinen las salidas de los otros iterativamente, llevando a un proceso de mejora continua

Escalabilidad: altamente escalables con mejora continua incorporada. Estos sistemas pueden evolucionar en tiempo real, haciéndose extremadamente efectivos para tareas dinámicas.

Complicaciones: Más complejo. Verificación de chequeos de cumplimiento (compliance checks) automatizados y autocorrección de acciones necesarias para asegurarse de que los agentes sigan alineados a las políticas mientras se adaptan.

Implementación (Patrones/Técnicas):

- Consensus
- Agent Negotiation
- Conflict Resolution
- Fractal CoT
- Coevolved Agent Training
- Trust Decay



Construir Agentes y Herramientas

Modelos



Direct String / Registro

Diseñado para modelos estrechamente integrados con Google Cloud.

- Gemini y Claude
- Agent Platform
- Gemma con Google AI Studio
- Resolución directa a través del registro interno de ADK



Conectores de Modelos

Para una mayor compatibilidad con modelos fuera de Google.

- Apigee y LiteLLM
- Ollama, vLLM y LiteRT-LM
- Utiliza clases wrapper personalizadas



Ruta de Modelos

Selección dinámica de modelos en tiempo de ejecución.

- Función router inteligente
- Conmutación por error (failover) automática
- Optimiza la disponibilidad y costos

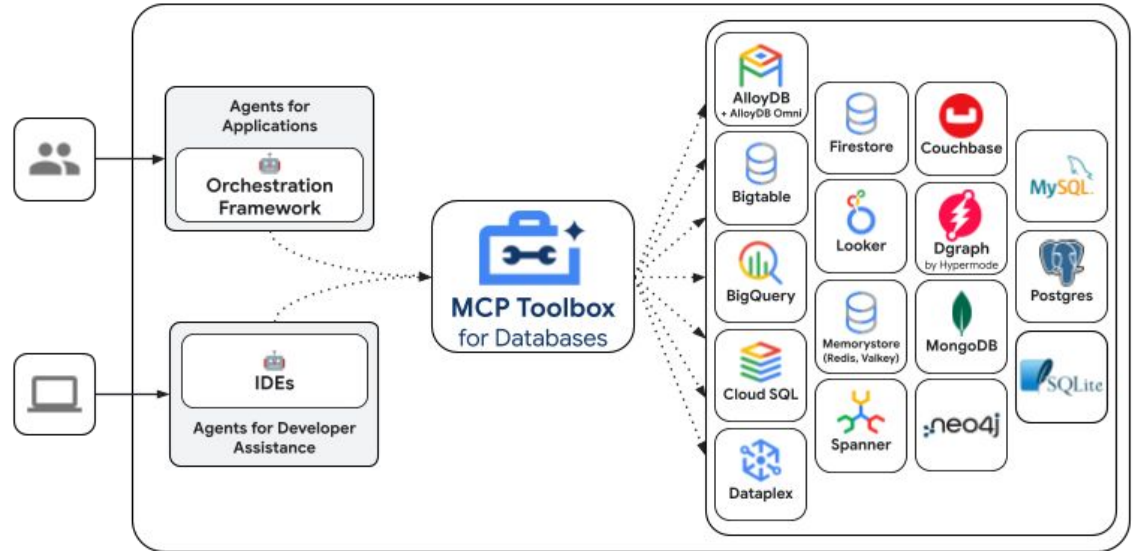
Langchain: <https://docs.langchain.com/oss/python/langchain/models>

Herramientas

Podemos usar herramientas para:

- Consultar bases de datos
- Hacer solicitudes API: obtener datos del clima, sistemas de reservas
- Buscar en la web
- Ejecutar fragmentos de código
- Recuperar información de documentos (RAG)
- Interactuar con otro software o servicios

y mucho más.



Herramientas

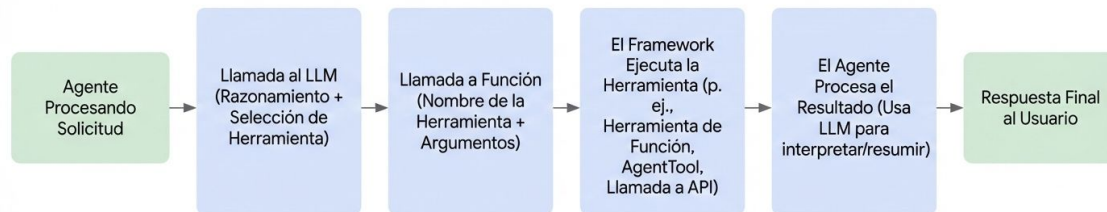
ADK trae una variedad de opciones para crear herramientas y para conectarse a otras ya creadas

La creación puede ser:

- [Function Tools](#)
- [Long Running Function Tools](#)
- [Agents-as-a-Tool](#)

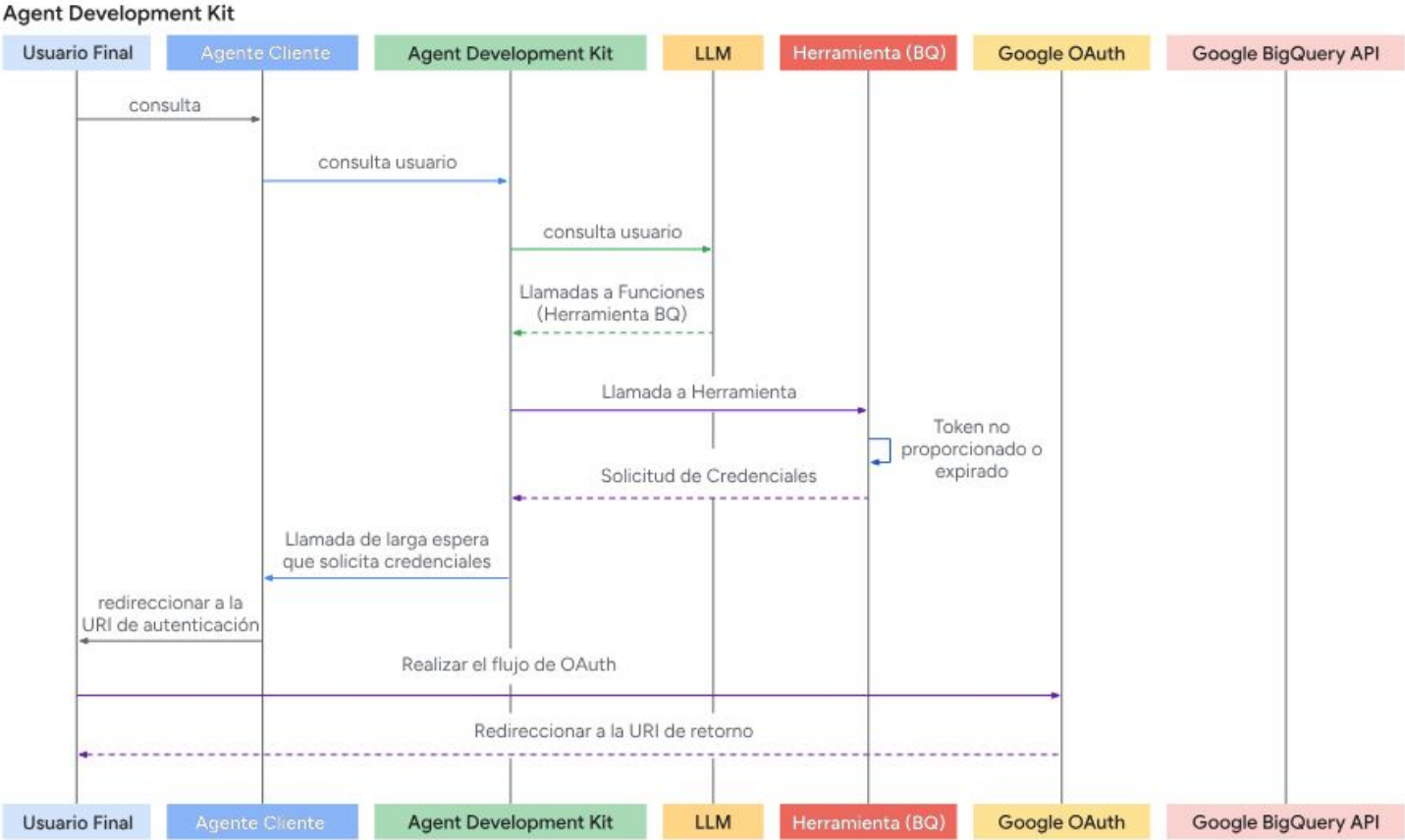
También puede usar:

- Herramientas de MCP
- Herramientas en formato OpenAPI

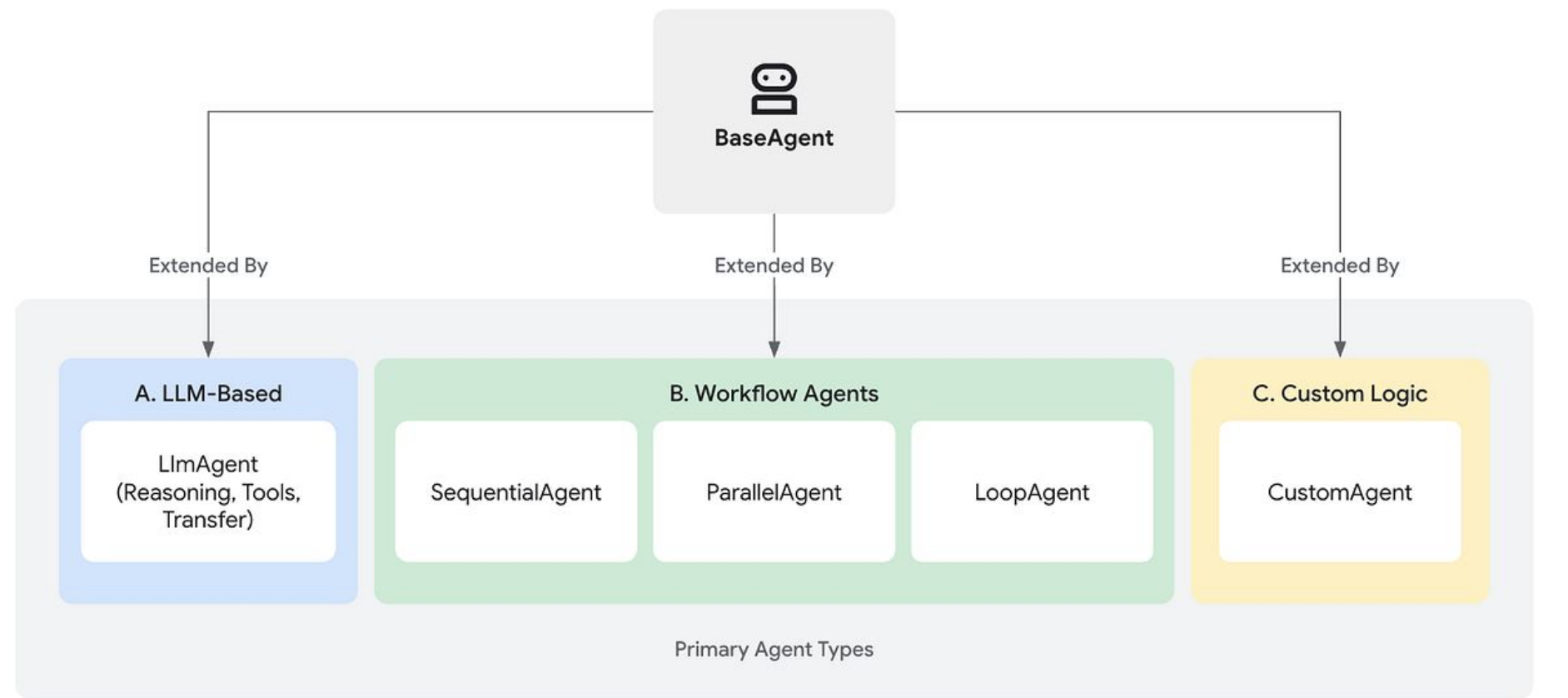


La autenticación va a ser importante, es decir, que el usuario tenga los permisos para ejecutar la herramienta. Para eso, se usa el protocolo OAuth

Proceso de Autenticación



Tipos de Agentes



LLM-Based

```
# Define una función de herramienta
def get_capital_city(country: str) -> str:
    """Recupera la ciudad capital para un país dado."""
    # Reemplaza con lógica real (ej., llamada API, búsqueda en base de datos)
    capitals = {"france": "Paris", "japan": "Tokyo", "canada": "Ottawa"}
    return capitals.get(country.lower(), f"Lo siento, no conozco la capital de {country}.")

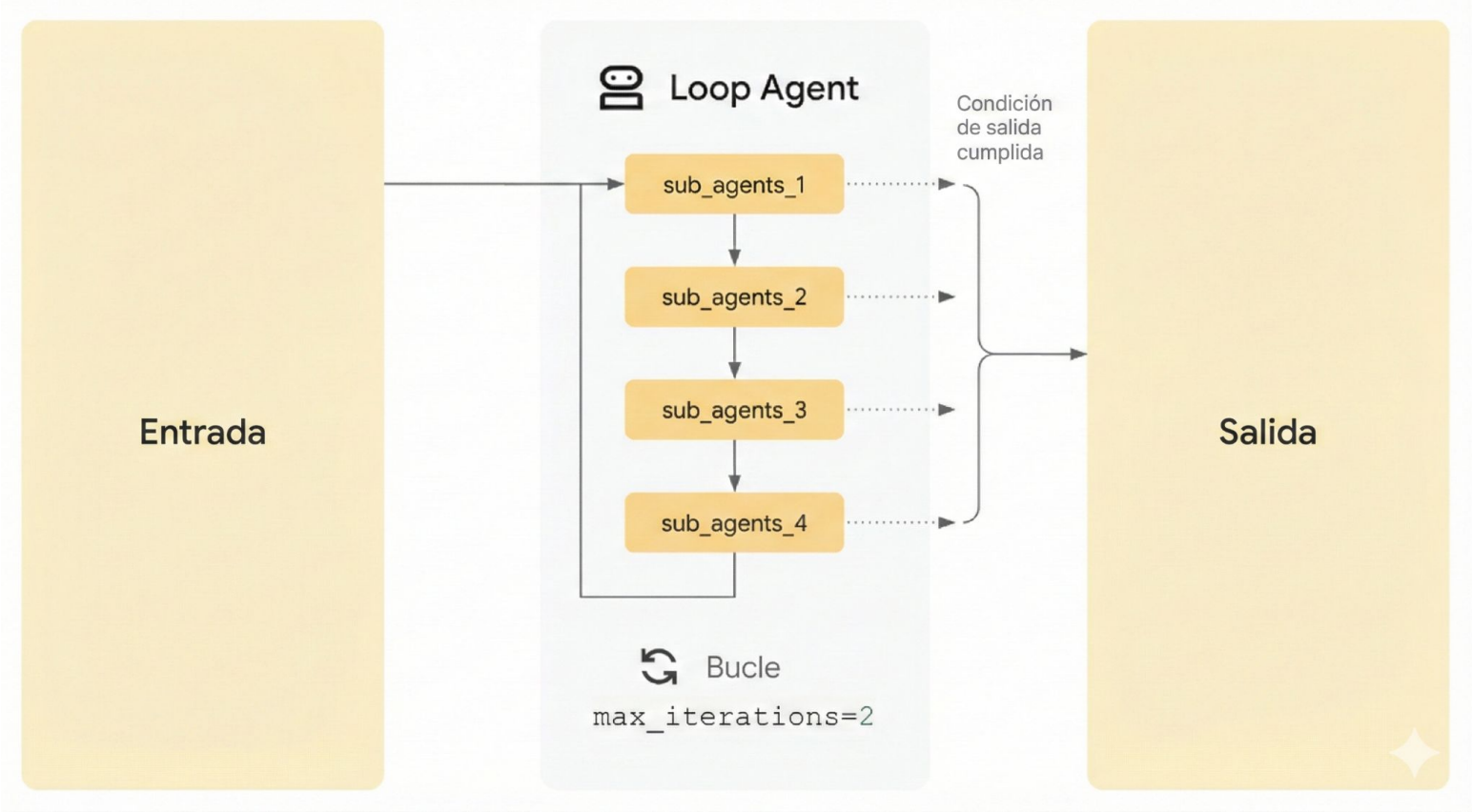
# Agrega la herramienta al agente
capital_agent = LlmAgent(
    model="gemini-2.5-flash",
    name="capital_agent",
    description="Responde preguntas de usuarios sobre la ciudad capital de un país dado.",
    instruction="""Eres un agente que proporciona la ciudad capital de un país... (texto de instrucción
anterior)""",
    tools=[get_capital_city] # Proporciona la función directamente
)
```

Sequential Agents

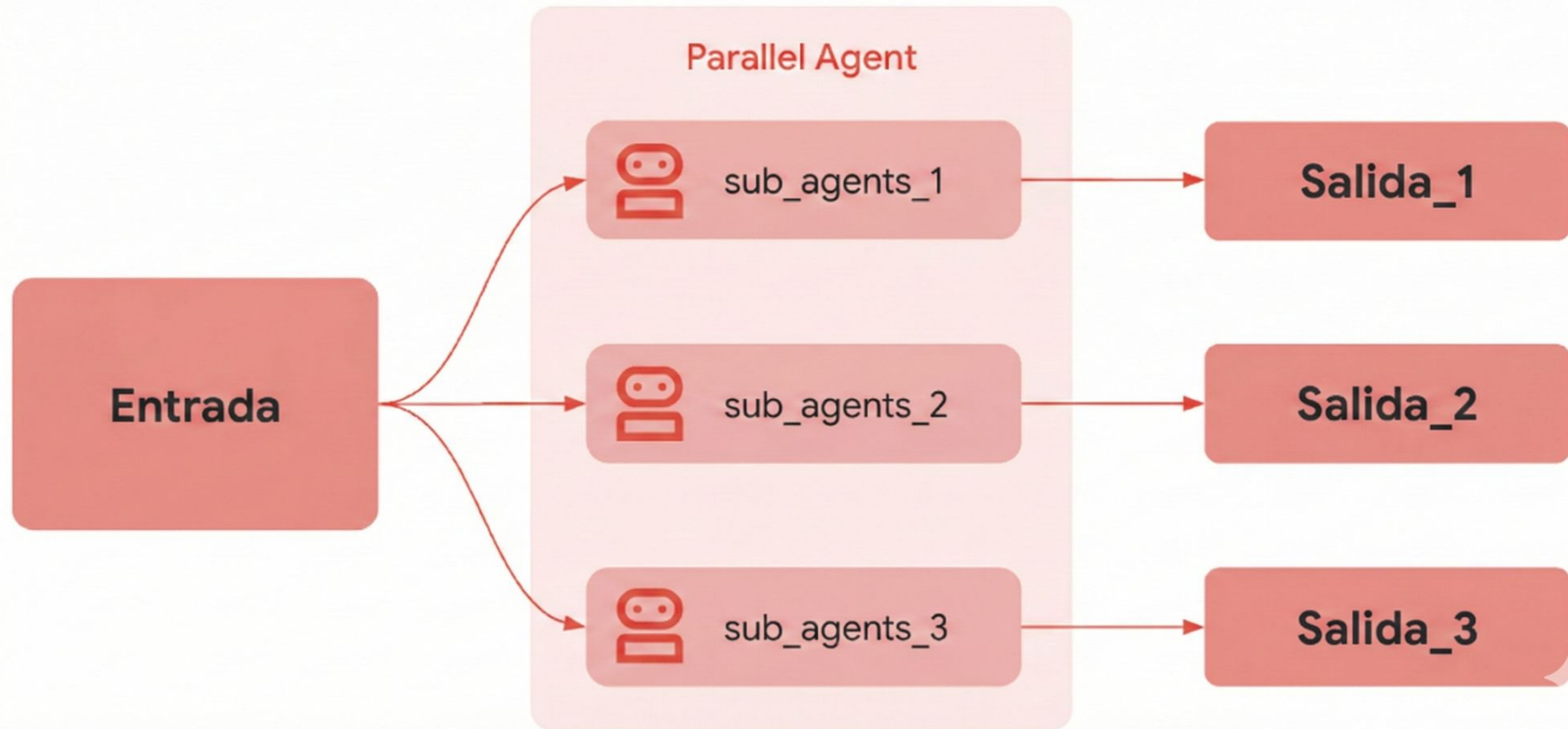


<https://adk-es.fabian-castro-c.dev/agents/workflow-agents/sequential-agents/#ejemplo>

Loop Agents



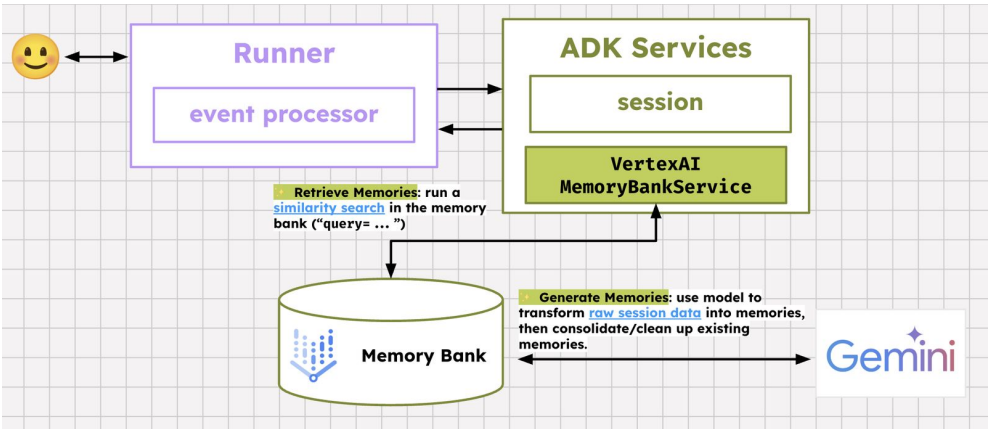
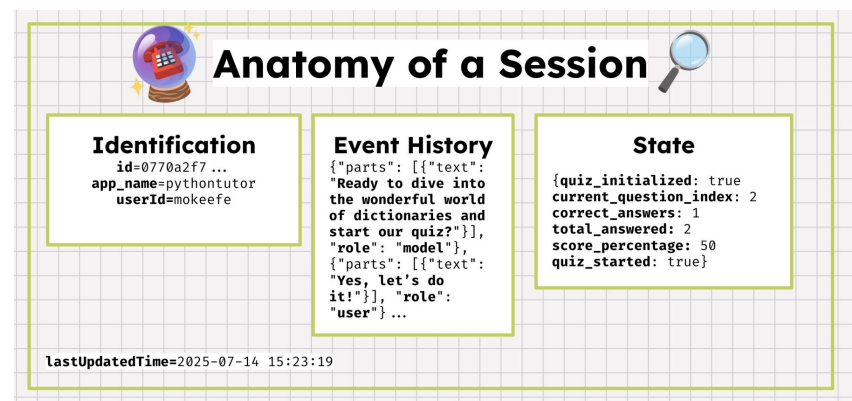
Parallel Agents



<https://adk-es.fabian-castro-c.dev/agents/workflow-agents/parallel-agents/#como-funciona>

Memoria

- **Session & State:** Se centran en la **interacción actual** – el historial y datos de la *única conversación activa*. Gestionados principalmente por un `SessionService`.
- **Memory:** Se centra en la **información pasada y externa** – un *archivo consultable* que potencialmente abarca múltiples conversaciones. Gestionado por un `MemoryService`.



<https://cloud.google.com/blog/topics/developers-practitioners/remember-this-agent-state-and-memory-with-adk>

Demo

Ejemplos de Agentes con Google ADK

<https://github.com/google/adk-samples>



Ejecutar Localmente y Depurar

ADK web

 Agent Development Kit

my_agent

+



Trace

Events

State

Artifacts

Sessions

Eval

Invocations

what can you do?

▼

what time is it in Paris?

▼

SESSION ID

a2580a79-ce58-4bfa-ab6f-f4725bf02a02

USER ID

user user

Token Streaming

+

New Session







I can tell you the current time in a specified city.



what can you do?



 get_current_time



 get_current_time



The current time in Paris is 10:30 AM.

what time is it in San Francisco?









Agent Development Kit

blogger_agent

+

Trace

State

Artifacts

Sessions

Eval

Invocations

plan a blog post about visiting tokyo

Invocation ID:
e-4c1f41f1-fd3c-4ff6-b539-c7e5ebf67930

|→ invocation

29599.06ms

invoke_agent interact...

29580.90ms

call_llm

29577.38ms

|→ generate_content...

29576.86ms

execute_tool

0.58ms

invoke_agent

27274.40ms

invokeAgen...

27252.08ms

call_llm

27236.55ms

|→ genera...

27236.31ms

invokeAgen...

10

SESSION ID 56bcabc1-9aee-42eb USER ID user Token Streaming + New Session

#1

plan a blog post about visiting tokyo

#2

⚡ transfer_to_agent

#3

✓ transfer_to_agent

Here is a blog post outline for visiting Tokyo:

The Ultimate Tokyo Adventure: Your G

I. Introduction

- **Welcome to Tokyo:** A city where ancient traditions beautifully intertwine v temples, and culinary delights.
- **What You'll Discover:** This guide will take you through Tokyo's iconic neig navigate this incredible metropolis.

II. Exploring Tokyo's Iconic Neighborhoods & Distric

Dive into the unique atmosphere of Tokyo's most famous areas, each offering a c

Type a Message...

ADK Web Developer UI

ADK web no es una herramienta productiva, vamos a tener que crearle una interfaz al agente.

Con lo cual, si queremos que otros usen el agente tendríamos que armar un frontend o integrarlo a una página web u otra aplicación.

Existen algunas librerías para crear interfaces de Agente iniciales: streamlit, gradio. También, se pueden hacer custom.

Vibe Coding & Agentic Engineering



Agent Skills

Es un simple formato abierto que permite extender y especializar con conocimientos los agentes. Se trata de una carpeta con un archivo markdown [SKILLS.md](#). También, se incluyen subdirectorios con información especializada.

```
my-skill/
├─ SKILL.md      # Required: metadata + instructions
├─ scripts/      # Optional: executable code
├─ references/   # Optional: documentation
├─ assets/       # Optional: templates, resources
└─ ...          # Any additional files or directories
```

Esta organización permite también reutilizar aquellas cosas que funcionaron.

Open Standard: <https://agentskills.io/home>

<https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>

Agent Skills - Formato de los archivos - SKILL.md

Consta de dos partes:

- **Frontmatter** en YAML
- **Cuerpo** en Markdown

Frontmatter en YAML

Esta es la capa de metadatos. Es la única parte de la habilidad que es indexada por el enrutador de alto nivel del agente. Cuando un usuario envía un *prompt*, el agente realiza una coincidencia semántica del *prompt* con los campos de descripción de todas las habilidades disponibles.

Campos clave:

- **name:** No es obligatorio. Debe ser único dentro del alcance (*scope*). Solo en minúsculas y se permiten guiones (por ejemplo, *postgres-query*, *pr-reviewer*). Si no se proporciona, tomará por defecto el nombre del directorio.
- **description:** Es obligatorio y es el campo más importante. Funciona como la "frase de activación". Debe ser lo suficientemente descriptivo para que el LLM (Modelo de Lenguaje Grande) reconozca su relevancia semántica. Una descripción vaga como "Herramientas de base de datos" es insuficiente. Una descripción precisa como *"Ejecuta consultas SQL de solo lectura contra la base de datos PostgreSQL local para recuperar datos de usuarios o transacciones. Usa esto para depurar estados de datos"* asegura que la habilidad sea seleccionada correctamente.

A simple SKILL.md file

pdf/SKILL.md

YAML Frontmatter

```
---
name: pdf
description: Comprehensive PDF toolkit for extracting text and tables,
merging/splitting documents, and filling-out forms.
---
```

Markdown

Overview

This guide covers essential PDF processing operations using Python libraries and command-line tools. For advanced features, JavaScript libraries, and detailed examples, see *./reference.md*. If you need to fill out a PDF form, read *./forms.md* and follow its instructions.

Quick Start

```
...
python
from pypdf import PdfReader, PdfWriter
```

```
# Read a PDF
reader = PdfReader("document.pdf")
print(f"Pages: {len(reader.pages)}")
```

```
...
```

A SKILL.md file must begin with YAML Frontmatter that contains a file name and description, which is loaded into its system prompt at startup.

<https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>

Agent Skills - Formato de los archivos - SKILL.md

El cuerpo en Markdown

El cuerpo contiene las instrucciones. Esto es "ingeniería de *prompts*" persistida en un archivo. Cuando se activa la habilidad, este contenido se inyecta en la ventana de contexto del agente.

El cuerpo debe incluir:

- **Goal (Objetivo):** Una declaración clara de lo que logra la habilidad.
- **Instructions (Instrucciones):** Lógica paso a paso.
- **Examples (Ejemplos):** Ejemplos de entradas y salidas (*few-shot*) para guiar el rendimiento del modelo.
- **Constraints (Restricciones):** Reglas sobre lo que "no se debe hacer" (por ejemplo, "No ejecutar consultas DELETE").

Si usan Antigravity or Antigravity CLI, se copia en
<project-root>/.agents/skills/ (project scope).

Global skills: ~/ .gemini/antigravity-cli/skills/.

A simple SKILL.md file

pdf/SKILL.md

```
YAML Frontmatter

name: pdf
description: Comprehensive PDF toolkit for extracting text and tables,
merging/splitting documents, and filling-out forms.

Markdown

## Overview

This guide covers essential PDF processing operations using Python libraries and command-line
tools. For advanced features, JavaScript libraries, and detailed examples, see ./reference.md.
If you need to fill out a PDF form, read ./forms.md and follow its instructions.

## Quick Start

```python
from pypdf import PdfReader, PdfWriter

Read a PDF
reader = PdfReader("document.pdf")
print(f"Pages: {len(reader.pages)}")

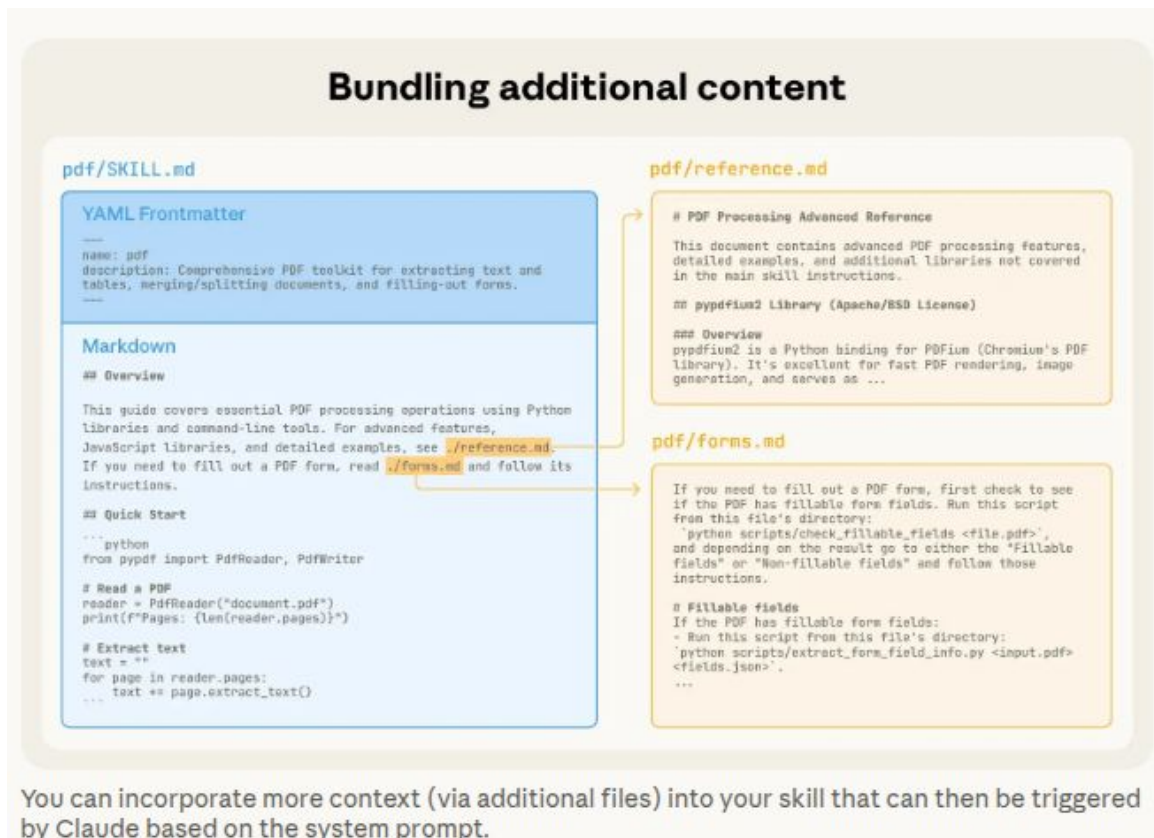
...

```

A SKILL.md file must begin with YAML Frontmatter that contains a file name and description, which is loaded into its system prompt at startup.

<https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>

# Agent Skills - Formato de los archivos - [SKILL.md](#) Referencias



<https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>

## Agent Skills - También se pueden incluir scripts

### Bundling executable scripts

pdf/forms.md

```
If you need to fill out a PDF form, first check
to see if the PDF has fillable form fields.
Run this script from this file's directory:
`python scripts/check_fillable_fields <file.pdf>`,
and depending on the result go to either the
"Fillable fields"
or "Non-fillable fields" and follow those
instructions.
```

```
Fillable fields If the PDF has fillable form
fields:
- Run this script from this file's directory:
`python ./extract_fields.py
<input.pdf> <fields.json>`.
...
```

pdf/extract\_fields.py

```
from pypdf import PdfReader

def write_field_info(pdf_path: str, output_path: str):
 """Extract form fields from PDF and store as JSON."""
 reader = PdfReader(pdf_path)
 fields = get_fields(reader)
 with open(output_path, "w") as f:
 json.dump(fields, f)

... omitted ...

if __name__ == "__main__":
 if len(sys.argv) != 3:
 print(f"Usage: python {sys.argv[0]} <pdf_path> <output_json_path>")
 sys.exit(1)
 write_field_info(sys.argv[1], sys.argv[2])
```

Skills can also include code for Claude to execute as tools at its discretion based on the nature of the task.

# Agent Skills - Progressive Disclosure

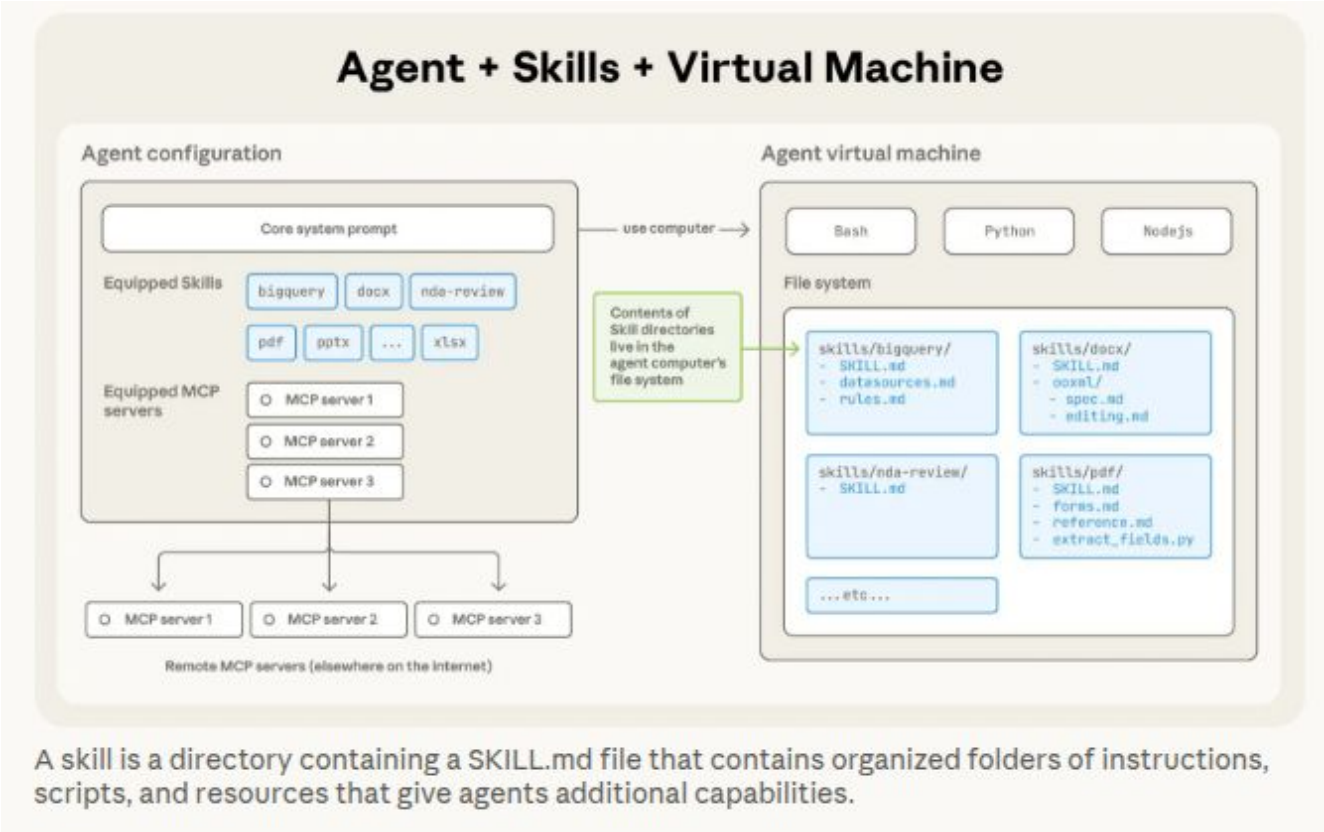
Los agentes cargan las habilidades a través de la revelación progresiva (*progressive disclosure*), en tres etapas:

- **Descubrimiento:** Al inicio, los agentes cargan solo el nombre y la descripción de cada habilidad disponible, lo justo para saber cuándo podría ser relevante.
- **Activación:** Cuando una tarea coincide con la descripción de una habilidad, el agente lee las instrucciones completas del archivo **SKILL.md** para incluirlas en su contexto.
- **Ejecución:** El agente sigue las instrucciones, ejecutando opcionalmente el código empaquetado o cargando los archivos de referencia según sea necesario.

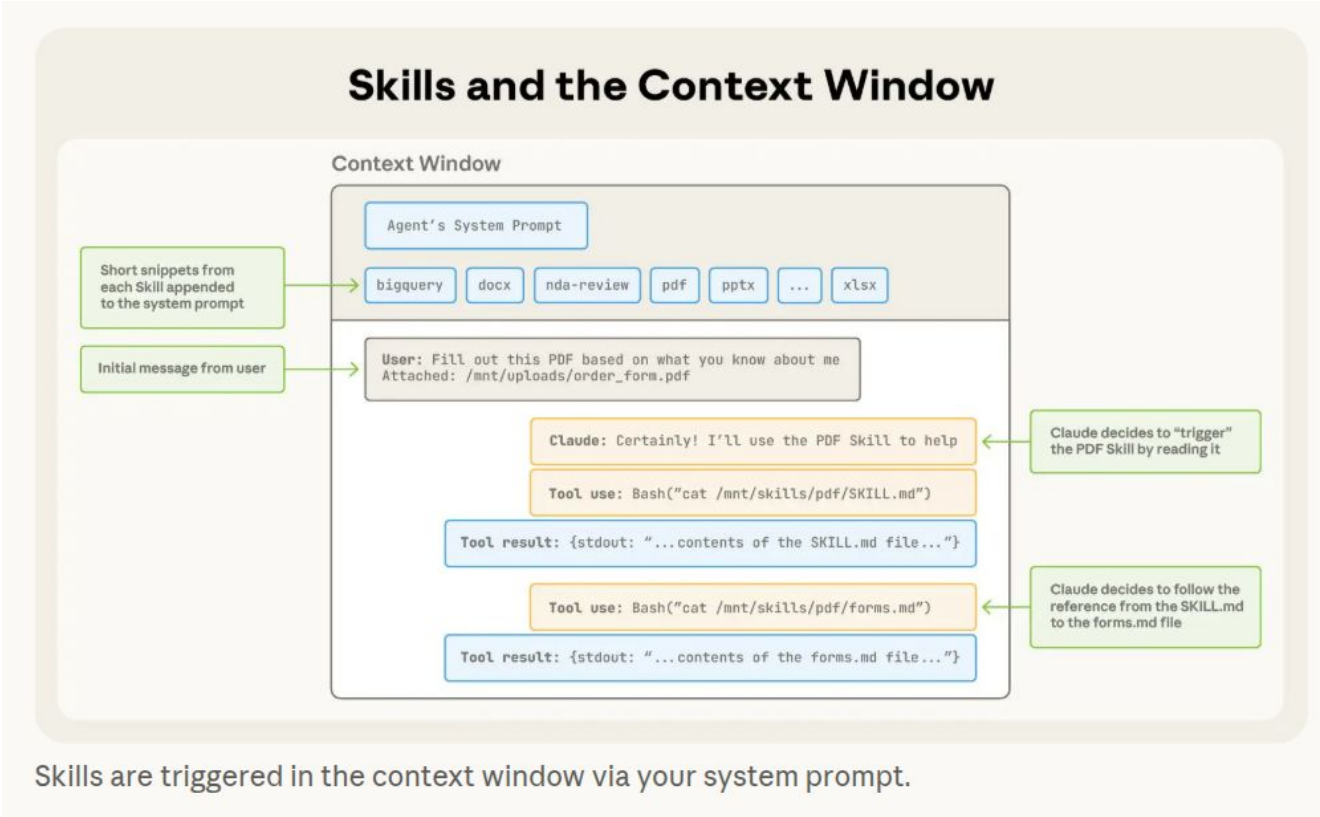
Esta organización permite también reutilizar aquellas cosas que funcionaron.

Level	File	Context Window	# Tokens
1	SKILL.md Metadata (YAML)	Always loaded	~100
2	SKILL.md Body (Markdown)	Loaded when Skill triggers	<5k
3+	Bundled files (text files, scripts, data)	Loaded as-needed by Claude	unlimited*

# Agent Skills - Cómo se ejecutan



# Agent Skills - Cómo se ejecutan

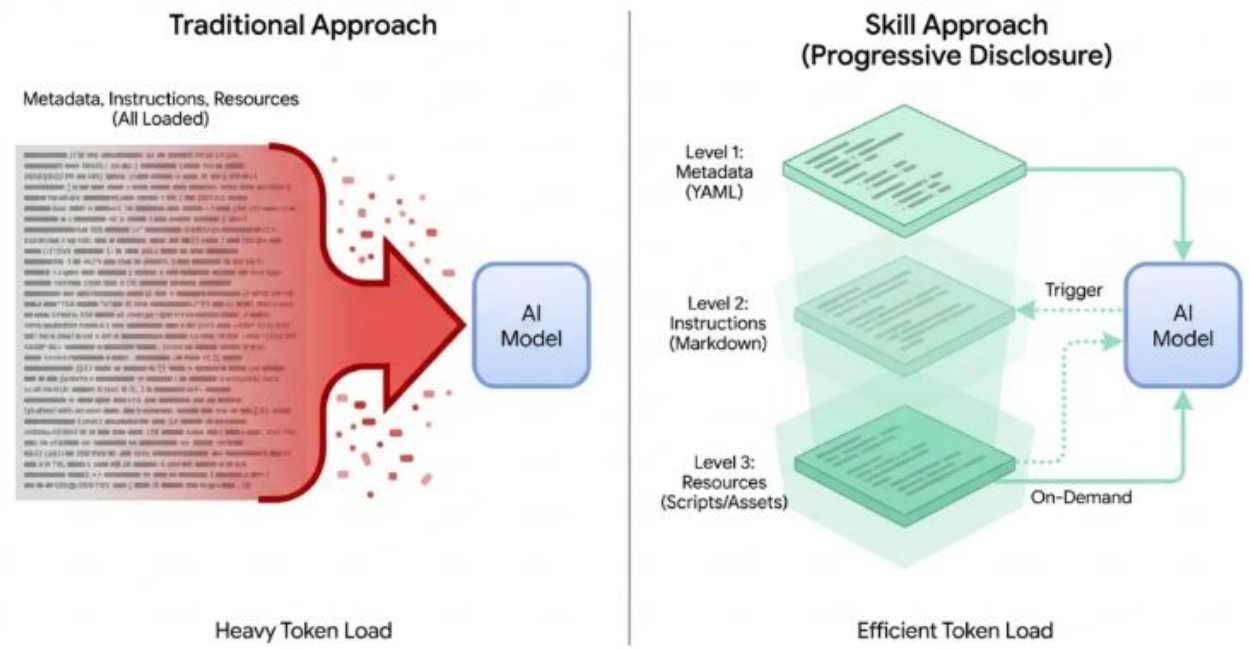




# Agent Skills

La revelación progresiva es la forma que se encontró de poder usar distintas habilidades sin incurrir en una saturación del contexto o tool bloat.

## Architecture of Efficiency: Progressive Disclosure vs. Static Context



The Progressive Disclosure model (right) drastically reduces initial token load compared to traditional context loading (left). Level 1 loads only lightweight metadata. Level 2 injects procedural instructions only upon trigger. Level 3 executes scripts and reads assets on-demand, keeping the context window streamlined.

Agent Skills: A Data-Driven Analysis of Claude Skills for Extending Large Language Model Functionality

George Ling<sup>1</sup>, Shanshan Zhong<sup>2</sup>, Richard Huang<sup>1</sup>  
<sup>1</sup>Bosch Research, <sup>2</sup>Carnegie Mellon University

Abstract

Agent skills extend large language model (LLM) agents with reusable, program-like modules that define triggering conditions, procedural logic, and tool interactions. As these skills proliferate in public marketplaces, it is unclear what types are available, how users adopt them, and what risks they pose. To answer these questions, we conduct a large-scale, data-driven analysis of 40,285 publicly listed skills from a major marketplace. Our results show that skill publication tends to occur in short bursts that track shifts in community attention. We also find that skill content is highly concentrated in software engineering workflows, while information retrieval and content creation account for a substantial share of adoption. Beyond content trends, we uncover a pronounced supply-demand imbalance across categories, and we show that most skills remain within typical prompt budgets despite a heavy-tailed length distribution. Finally, we observe strong ecosystem homogeneity, with widespread intent-level redundancy, and we identify non-trivial safety risks, including skills that enable state-changing or system-level actions. Overall, our findings provide a quantitative snapshot of agent skills as an emerging infrastructure layer for agents and inform future work on skill reuse, standardization, and safety-aware design.

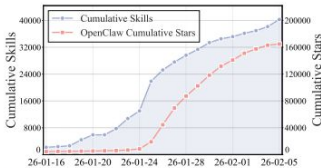


Figure 1: **Growth trend of agent skills.** According to the well-known agent skills platform *skills.sh*, the number of recorded skills experienced rapid growth from mid-January to early February 2026, exceeding 40,000 by early February. During the same period, the popular open-source skills application OpenClaw (*openclaw Community, 2026*) saw a sharp surge in GitHub stars, reaching over 25,000 stars in a single day at the end of January, followed by a gradual decline, with the total number of stars exceeding 170k.

et al., 2023; Sapkota et al., 2025). This agentic execution model has enabled a growing range of applications, from software development and data analysis to personal assistance and workflow automation (Wang et al., 2024).

As agent-based systems scale in both complexity and deployment, new challenges arise around reliability, reuse, and maintainability (Liu et al., 2026a; Cemri et al., 2025; Raheem and Hossain, 2025). Many agent behaviors recur across tasks and users, yet are repeatedly re-specified through

Las habilidades (*skills*) son procedimientos ejecutables que interactúan con servicios externos, entornos locales y el contexto del usuario. En comparación con las interacciones que dependen únicamente de *prompts*, estas expanden la superficie de riesgo (o superficie de daño) porque una habilidad puede acceder a datos sensibles o desencadenar efectos secundarios en el mundo real.

# Demo

## Usando Skills

### en Coding Agentes

(claude code, antigravity)

# Agent Skills - Mejores prácticas, Evaluación y Optimización

Mejores prácticas: <https://agentskills.io/skill-creation/best-practices>

Optimización de Skills: <https://agentskills.io/skill-creation/optimizing-descriptions>

Evaluación de Skills: <https://agentskills.io/skill-creation/evaluating-skills>

Ejemplos de Skills: <https://github.com/rominirani/antigravity-skills>

Tutorial: <https://platform.claude.com/cookbook/skills-notebooks-01-skills-introduction>

Usar skills pre-made anthropic: <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/quickstart>

The Complete Guide to Building Skills for Claude: <https://resources.anthropic.com/hubfs/The-Complete-Guide-to-Building-Skill-for-Claude.pdf>

Patrones para Skills ADK: <https://lavinigam.com/posts/adk-skill-design-patterns/>

NPX Skills: <https://github.com/vercel-labs/skills>

Coding in Claude Code: <https://code.claude.com/docs/en/features-overview#subagent-vs-agent-team>

Custom subagents: <https://code.claude.com/docs/en/sub-agents>

Claude Agents: <https://code.claude.com/docs/en/agents>

Cookbooks Claude: [https://github.com/anthropics/claude-cookbooks/blob/main/managed\\_agents/CMA\\_plan\\_big\\_execute\\_small.ipynb](https://github.com/anthropics/claude-cookbooks/blob/main/managed_agents/CMA_plan_big_execute_small.ipynb)

## Día 3: Temas que vimos...

- Fundamentos teóricos de los agentes de IA, su ciclo iterativo de pensamiento, acción y corrección, y ejemplos de aplicaciones en el mercado como herramientas de programación, chatbots y sistemas IVR.1
- Arquitecturas multiagente, niveles de maduración de los sistemas agénticos y patrones de coordinación complejos.1
- Frameworks de desarrollo destacados como LlamaIndex, LangChain y Google ADK.
- Seguridad en agentes: prevención de inyecciones, medidas de protección y buenas prácticas.1
- Agent Skills: estándares, formato, revelación progresiva y optimización de habilidades.